HealthRetailers data warehouse

Complete implementation guide

English edition | Final implemented scope | 28 October 2026

From dirty source exports to a tested SQL Server warehouse

This guide explains the complete implemented project so that a group member can understand, demonstrate and rerun it. It follows the actual data through file preparation, staging, dimensions, facts and validation. Technical terms are defined as they appear and collected in a glossary.

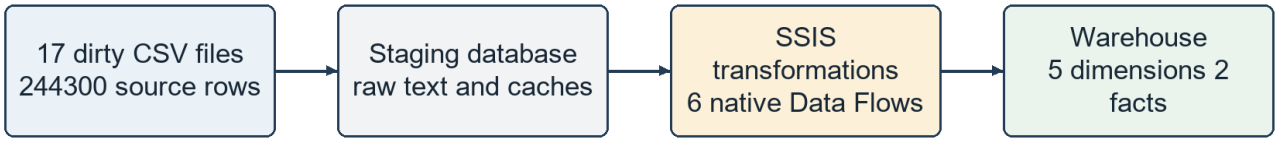


Figure 1. The implemented pipeline at a glance.

The delivered scope is five dimensions and two facts: current stock and validated supplier invoice lines. All 67 recorded checks passed. The guide distinguishes raw evidence, derived information, known unknowns and out-of-scope business processes.

What is ready	Verified result
Raw source integration	17 selected exports and 244300 staged rows
Dimension foundation	Product client supplier payment mode calendar
Fact loading	6842 stock products and 6964 purchase lines
Purchase coverage	880 validated invoices with exact net-header reconciliation
Technical evidence	Native package runs stable reruns tested repairs trusted FKs

Reading copies: this PDF and the editable Word document contain the same guide. The operational package and SQL files remain the execution authority.

Contents

1 Project purpose and final scope 4
2 Source files and data coverage 5
3 What dirty data means in this project 6
4 Architecture and the role of each tool 7
5 One time setup and repeated execution 8
6 Preserve the originals and prepare transport 9
7 Raw loading into the Staging Area 10
8 Master package orchestration 11
9 Inside the warehouse loading package 12
10 The final warehouse model 13
11 Grain keys and measurable quantities 14
12 Product dimension Data Flow 15
13 How product descriptions are reconstructed 16
14 Client dimension Data Flow 17
15 Supplier dimension Data Flow 18
16 Payment mode dimension Data Flow 19
17 Time dimension and the business calendar 20
18 Shared transformation rules 21
19 Prepare the facts before loading 22
20 Current stock Data Flow 23
21 Understand stock measures and unknown dates 24
22 Construct purchase candidates from staging 25
23 Purchase selection and dimension lookups 26
24 Native insertion and update decisions 27
25 Calculate net purchase amounts correctly 28
26 Rounding and purchase field lineage 29
27 What happens when the load runs again 30
28 Validation and the meaning of 67 checks 31
29 Final SQL assertions and recorded results 32
30 Run the project on the current machine 33
31 Configure or move the project 34
32 Analyse the warehouse in SSMS 35
33 Diagnose a failed execution 36
34 Known limits and responsible interpretation 37

35 Explain the project to the teacher 38
36 Implementation files and traceability 39
Appendix A Warehouse field dictionary 1 40
Appendix A Warehouse field dictionary 2 41
Appendix A Warehouse field dictionary 3 42
Appendix A Warehouse field dictionary 4 43
Appendix B Raw source fields 1 44
Appendix B Raw source fields 2 45
Appendix B Raw source fields 3 46
Appendix B Raw source fields 4 47
Appendix B Raw source fields 5 48
Appendix C Recorded checks 1 49
Appendix C Recorded checks 2 50
Appendix C Recorded checks 3 51
Appendix C Recorded checks 4 52
Appendix D Technical terms 1 53
Appendix D Technical terms 2 54
Appendix D Technical terms 3 55
Appendix D Technical terms 4 56
Appendix D Technical terms 5 57
Appendix D Technical terms 6 58
Appendix D Technical terms 7 59
Appendix D Technical terms 8 60
Appendix D Technical terms 9 61
Appendix D Technical terms 10 62
Appendix E Evidence and reading order 63

1 Project purpose and final scope

HealthRetailers is a retail data warehouse project. A data warehouse is a database organized for analysis across business activities and time. The final implementation imports dirty production exports, preserves their values in staging, derives consistent business descriptions and loads a warehouse with five dimensions and two fact tables.

A dimension describes an analysis subject, such as a product or supplier. A fact table records measurements at a precisely defined level of detail. Here, stock records current quantities by product, while purchases record quantities and net amounts by validated supplier invoice line.

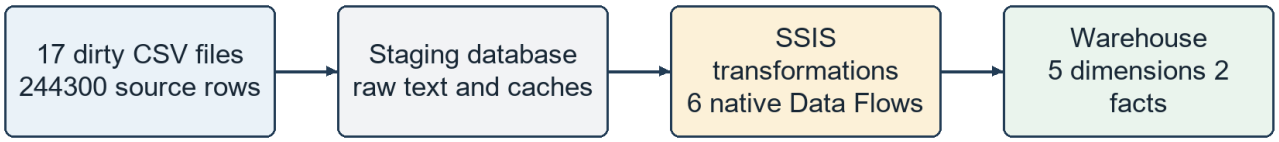


Figure 2. The complete implemented journey. The 244300 rows belong to the 17 sources used by this version, not to every file in the wider original repository.

How the project reached this version

The original DataWarehouseProject_3AI4 repository was cloned with its remote branches available. Its wider dirty-data collection contains 29 source exports. An earlier, broader warehouse implementation remains in that separate repository. It is not called by the final Master package described here.

The working base then switched to Amal's HealthRetailers_DW repository. Her two-package staging and dimension structure, four dimension diagrams, Sort and Merge Join patterns, and Lookup patterns were retained. Local corrections addressed configuration, text handling, product matching and NULL comparisons. Two fact Data Flows were then added inside her ETL_SA_DW package.

The final deliverable is the local HealthRetailers_DW_amal project on branch codex/fix-five-dimensions. No claim is made that Amal's remote GitHub repository was updated. Customer sales, receipts, supplier payments, machine learning and additional facts are outside this final two-fact scope.

2 Source files and data coverage

The sources are CSV exports from the production application. CSV means comma-separated values, but these files use either commas or semicolons according to their connection definition. The solution reads files; it does not connect directly to a live production database or call a production API.

Source export	Staging table	Rows	Decoding
Clients - 24112025081756 (3).csv	Clients_SA	93	UTF8
CompaniesList.csv	CompaniesList_SA	156	CP1252
documentItemm.csv	DocumentItem_SA	38,705	UTF8
Documents.csv	Documents_SA	12,542	CP1252
DocumentTypes.csv	DocumentTypes_SA	15	UTF8
Events.csv	Events_SA	82,585	UTF8
Fournisseurs - 24112025081816 (2).csv	Fournisseurs_SA	118	UTF8
PaymentsList.csv	PaymentsList_SA	10,280	UTF8
Payment.csv	Payment_SA	10,899	CP1252
ProductBrands.csv	ProductBrands_SA	212	UTF8
ProductStocks.csv	ProductStocks_SA	6,842	UTF8
Produits - 24112025081625 (2).csv	Produits_SA	7,800	UTF8
RelatedDocuments.csv	RelatedDocuments_SA	864	UTF8
StockMovements.csv	StockMovements_SA	49,609	UTF8
TransactionPayments.csv	TransactionPayments_SA	10,962	UTF8
Transactions.csv	Transactions_SA	12,523	UTF8
users_settings_csv.csv	UsersSettings_SA	95	UTF8
TOTAL	17 raw staging tables	244,300	

The manifest records the source filename, delimiter, decoding used, target staging table, ordered columns, row count and hashes. Appendix B lists the fields retained from each source. Rows are not globally deduplicated or deleted during transport.

Source row counts are not expected to equal dimension counts. For example, 93 customer-export rows help identify 82 customer dimension members through company identifiers and business rules. Members include explicit unknown and anonymous entries. A dimension is a conformed description at its own grain, not a copy of one CSV.

3 What dirty data means in this project

Dirty data is information that needs interpretation, normalization or quality checks before it can support reliable analysis. Dirty does not mean every row is invalid. The aim is to preserve evidence, apply explainable rules and expose what cannot be recovered.

Observed issue	Why it matters	Treatment
Mixed UTF8 and CP1252 files	The same bytes can display different characters under the wrong encoding.	Decode the original correctly and create Unicode transport.
Quotation and JSON text	Incorrect quote settings retain extra quotation marks or split a cell.	Parse and write CSV with consistent quoting; enable the SSIS text qualifier.
Amounts and dates stored as text	Text cannot safely be summed or compared as a date.	Use explicit numeric conversion and month day year date parsing.
Missing names or links	An invoice or product can lack a reliable description.	Use documented fallbacks or an explicit unknown member.
HTML in descriptions	Markup is not the intended product label.	Strip tags and decode selected HTML entities for product matching.
Repeated or damaged product codes	Different catalogue records can share an unreliable code.	Retain distinct attribute variants and require stronger matching for ambiguity.
Drafts and comment lines	They are source records but not completed purchases.	Keep raw rows; route excluded purchase candidates to Rejets_Faits.
NULL values	A normal comparison can return unknown instead of true or false.	Use explicit NULL-safe change detection.

Mojibake is the damaged text produced by reading bytes with the wrong character encoding. A replacement character, written as U+FFFD, means an earlier conversion could not represent the original character. Known mojibake sequences can be repaired; an unknown original letter cannot be recovered with certainty just by guessing.

Examples in this guide using Product A or Supplier X are fictional teaching examples. The operational totals and row counts are measured results from the implementation.

4 Architecture and the role of each tool

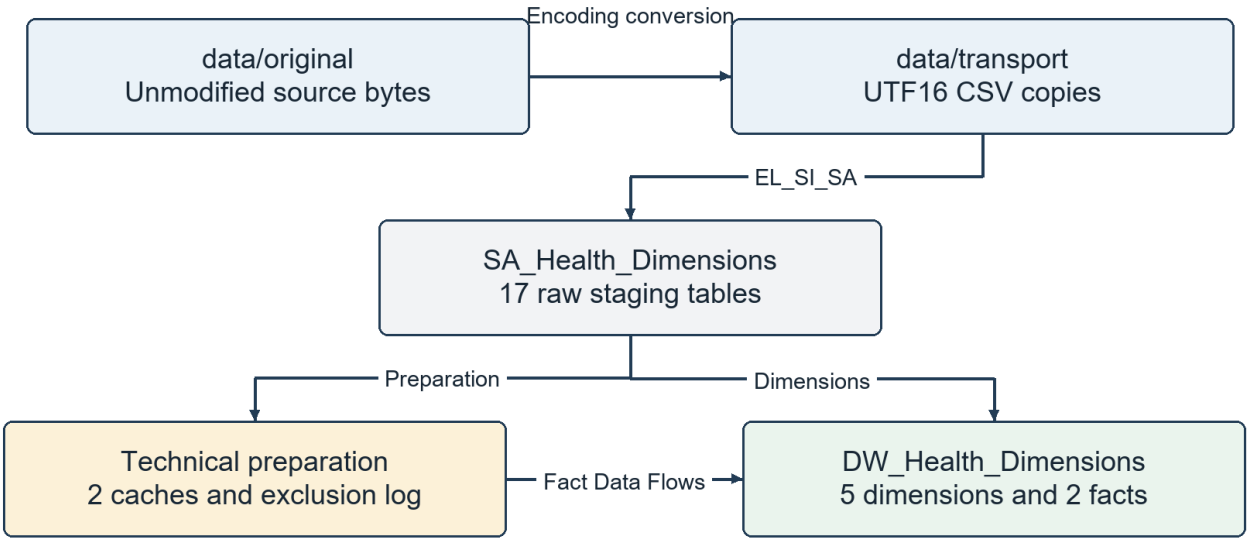


Figure 3. The physical layers. The staging database contains the raw source tables and technical preparation objects. Both databases run on the local SQL Server instance.

SQL Server 2022 Developer is the database engine: it stores tables, executes SQL, enforces keys and supplies data to SSIS. SA_Health_Dimensions is the staging database. DW_Health_Dimensions is the analysis database; its historical name still contains Dimensions although it now holds two facts as well.

SSMS, SQL Server Management Studio, is a client for administering SQL Server and running queries. It is not the database itself and it does not need to stay open while packages execute.

Visual Studio 2022 with the SSIS Projects extension opens the solution and its visual package designers. SSIS, SQL Server Integration Services, is the integration runtime that executes package tasks and Data Flows. The full SSIS 2022 runtime was added after the original environment was found to lack that feature.

PowerShell, Python and the SSIS authoring API were used to configure and generate compatible files without operating the user’s screen. The saved package components are native SSIS components, not a custom executable pretending to be a Data Flow. DTExec executes the compiled project outside the designer.

5 One time setup and repeated execution

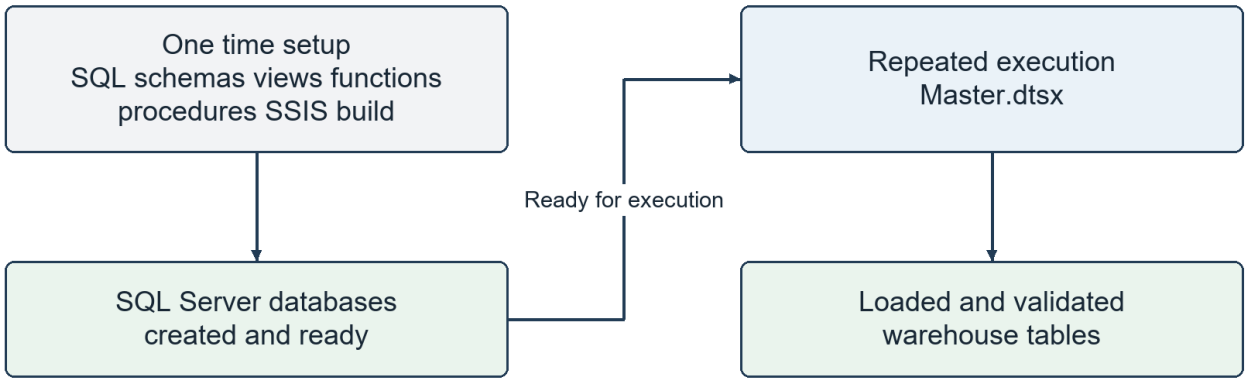


Figure 4. Database creation is a setup operation. Data loading is a repeatable package execution. SQL may participate in both, but SSMS is not a required manual step for each load.

The one-time setup creates the two databases, the five dimension tables, the two fact tables, staging tables, functions, views and procedures. It also configures local connection strings and builds the SSIS project. This setup has already been completed on the current machine.

The scripts can be executed in SSMS with SQLCMD Mode enabled, but the supplied setup.ps1 runs them through sqlcmd. SQLCMD variables substitute the chosen database names into scripts before SQL execution. A script is a saved set of commands; a stored procedure is a named routine installed inside a database.

Step	What performs it	When
Create database objects	SQL scripts called by setup.ps1	Initial setup or an intentional schema deployment
Prepare transport and connections	configure.py	Setup or a deliberate source/configuration refresh
Load raw sources	EL_SI_SA.dtsx	Every Master execution
Load calendar and dimensions	Master SQL task and ETL_SA_DW	Every Master execution
Prepare and load the facts	Preparation task and two native Data Flows	Every Master execution
Validate the warehouse	Master Verification SQL task	After successful loading

After setup, run Master.dtsx. There is no requirement to manually execute calendar SQL or insert warehouse rows in SSMS before each run. Reading and analysing the resulting tables in SSMS is a separate activity.

6 Preserve the originals and prepare transport

The data/original folder holds the 17 selected source files byte for byte. configure.py reads each file, attempts UTF8 with optional BOM first, then falls back to CP1252 and CP850 if decoding requires it. In the measured manifest, 14 files decoded as UTF8 and three as CP1252. This is an implementation-specific decoding strategy, not a universal encoding detector.

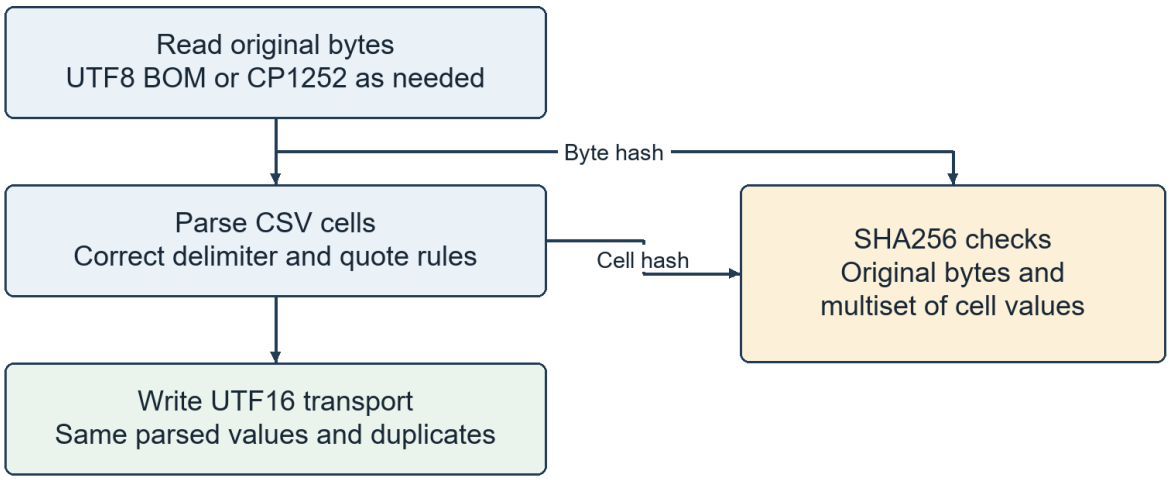


Figure 5. Transport conversion changes representation, not the parsed business values. Hashes provide separate evidence for original file bytes and loaded cell contents.

The script parses records with the configured separator and checks the number of fields. If that separator does not match the header width, it tries the other supported separator. A remaining column mismatch fails configuration rather than padding away unexplained data.

Parsed records are written to data/transport as UTF16 CSV with consistent CRLF record separators and quotation handling. Embedded quotation marks are escaped by the CSV writer. The SSIS flat-file managers use Unicode and a double-quote text qualifier. Every flat-file column is text-qualified.

Unicode is a character representation that supports accented letters and many writing systems. UTF8 and UTF16 are encodings of Unicode. Changing to UTF16 does not repair already damaged source letters; it prevents a new encoding mismatch during this import. The raw files remain available for investigation.

7 Raw loading into the Staging Area

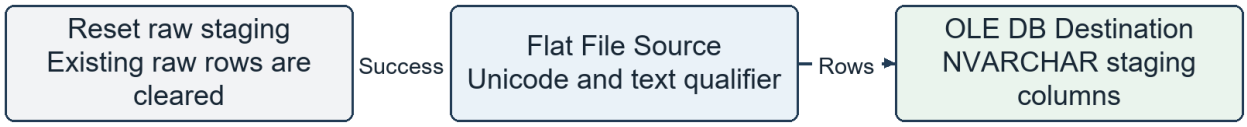


Figure 6. One representative raw loading path. EL_SI_SA contains 17 corresponding Flat File Source to OLE DB Destination Data Flows after its staging reset.

Staging Area, abbreviated SA, means the intermediate database used to receive source data before warehouse loading. The raw staging tables use NVARCHAR(4000) for text fields, with NVARCHAR(MAX) for the long UsersSettings Value field. NVARCHAR stores Unicode text; MAX allows the larger text content needed here.

Keeping numeric-looking values as text at this stage is deliberate. An amount with a comma, a blank supplier identifier or an unusual code is preserved before interpretation. Raw staging is not the final typed warehouse and is not globally cleaned in place.

EL_SI_SA first clears the raw staging tables, then imports the configured transport files. This is a full refresh of the source staging snapshot. It does not mean that every warehouse table is truncated. Warehouse dimensions and facts use the insertion and update paths described later.

Input	Operation	Output
Transport CSV	Flat File Source reads delimited, quoted Unicode values.	An SSIS row with string columns
SSIS row	OLE DB Destination maps the source columns to SQL columns.	A row in the corresponding *_SA table
All source rows	Count and cell-hash checks compare the source and SQL contents.	Evidence of faithful transfer, including duplicates

The 244300 loaded source records include data used for customer and payment dimensions as well as purchases and stock. Only a subset is eligible for the purchase fact. A source record may be preserved in staging without becoming a fact.

8 Master package orchestration

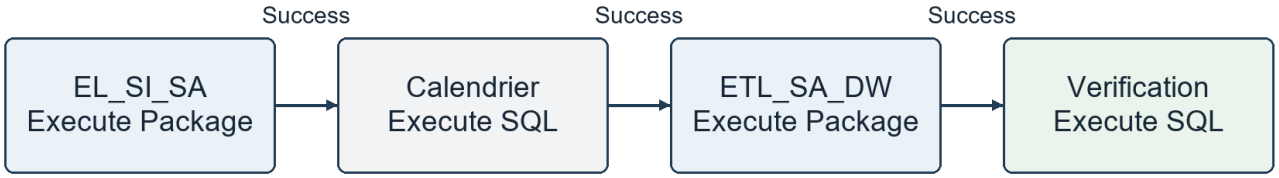


Figure 7. Actual Master task order. Each connector is a success dependency: the next task runs only after its predecessor succeeds.

A package is a saved SSIS workflow in a .dtsx file. A Control Flow describes which tasks execute and in what order. A precedence constraint is the connector that controls execution according to a condition, here success. A Data Flow describes how rows move through sources, transformations and destinations inside a task.

What each Master task does

EL_SI_SA is an Execute Package task that runs the raw import package. Calendrier is an Execute SQL task that fills missing dates in dim_temps. ETL_SA_DW is another Execute Package task that runs the dimension and fact Data Flows. Verification is an Execute SQL task containing the warehouse assertions.

Amal’s original Master called staging and dimensions in sequence. Calendar loading and final verification were added so that the final entry point runs the entire implemented workflow without requiring a separate manual calendar step.

A check that throws a SQL error causes the validation task and package execution to fail. A green task means that task completed successfully; it does not mean every source description was recoverable or every future scenario has been tested.

To see row transformations, open EL_SI_SA or ETL_SA_DW and double-click a Data Flow task. Master is the coordinator, which is why its diagram is intentionally much simpler than the dimension or purchase diagrams.

9 Inside the warehouse loading package

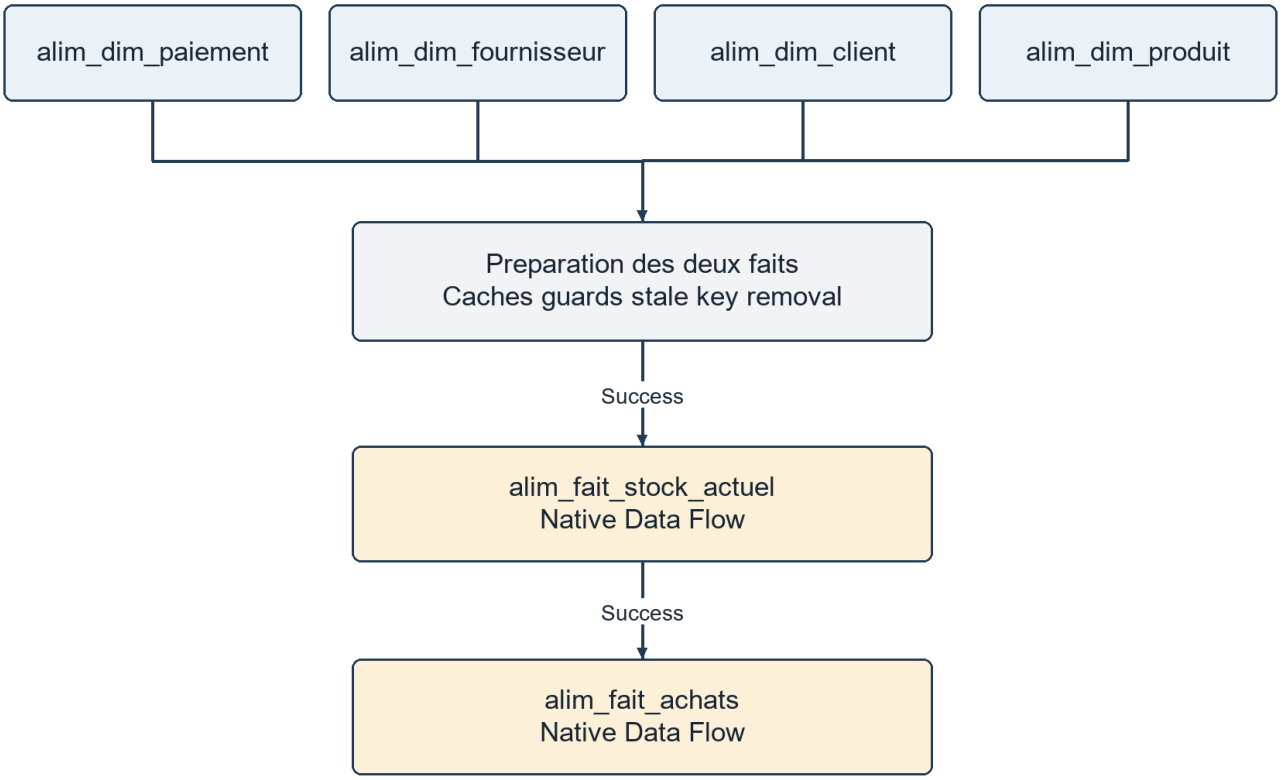


Figure 8. The implemented Control Flow inside ETL_SA_DW. Four original dimension tasks precede preparation, current stock and purchases.

The four dimension tasks have independent inputs at the Control Flow level, but this package sets MaxConcurrentExecutables to 1. Therefore they execute one at a time; the diagram is not a promise of parallel execution. All four must finish before fact preparation can begin.

The package contains six native Data Flows: alim_dim_client, alim_dim_fournisseur, alim_dim_paiement, alim_dim_produit, alim_fait_stock_actuel and alim_fait_achats. dim_temps is the fifth dimension, but it is generated by the Master SQL task rather than a CSV Data Flow.

Preparation des deux faits rebuilds technical caches, checks mandatory source values, clears the exclusion log and removes fact keys that are no longer eligible. The stock Data Flow then completes before the purchases Data Flow runs. SQL preparation does not replace their native warehouse INSERT and UPDATE paths.

10 The final warehouse model

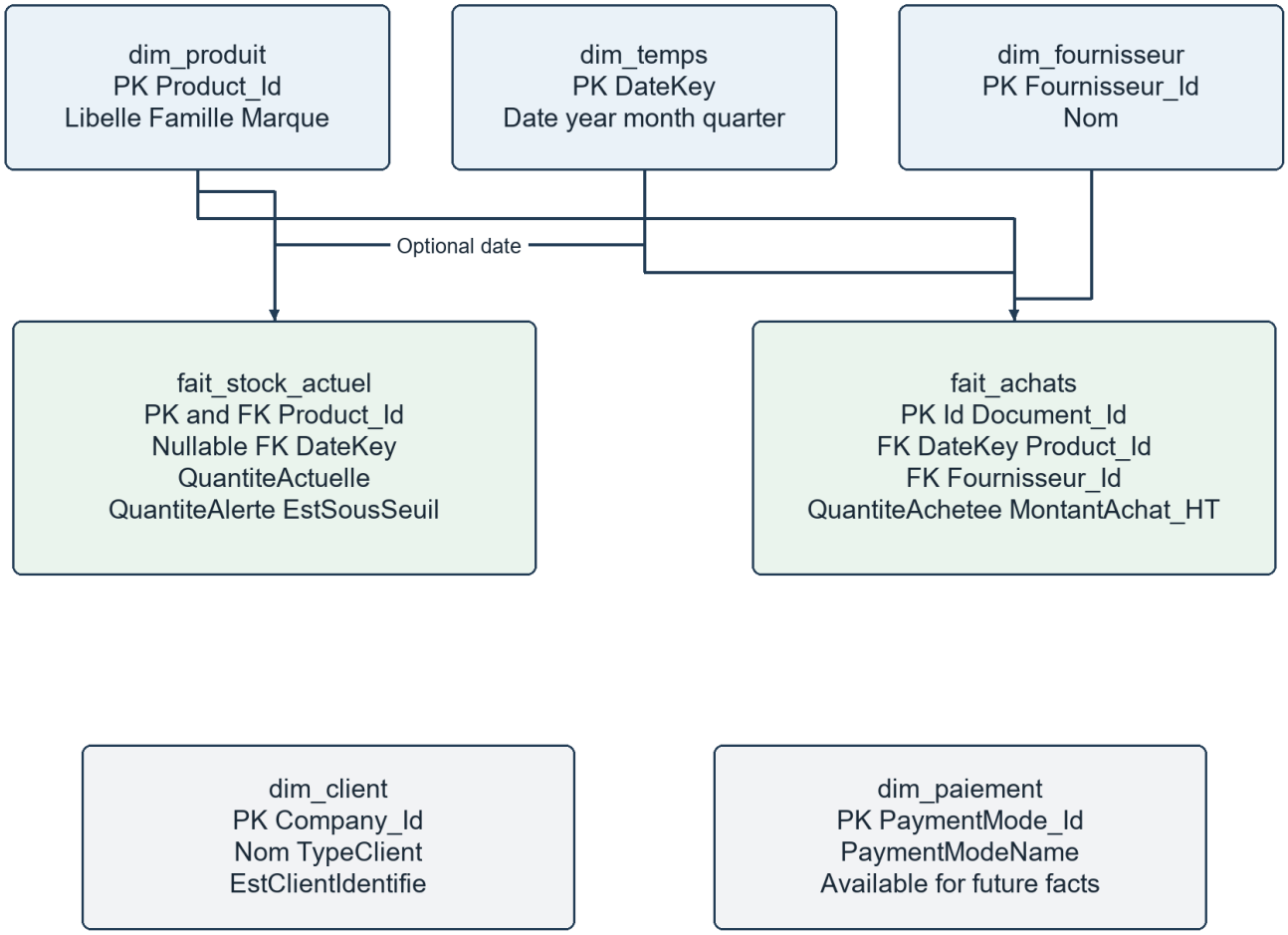


Figure 9. The implemented seven-table model. Lines represent foreign-key relationships. Product and date are shared dimensions. Client and payment are available dimensions without a foreign-key link from these two facts.

A star schema places a fact table at the centre of its descriptive dimensions. Because these two facts share product and date dimensions, the overall design can also be described as a small fact constellation. Shared dimensions allow consistent product and period analysis across business processes.

Stock has one current row per product, not a history of daily captures. Its DateKey is nullable because the export does not prove a capture date. Purchases has one row per validated supplier invoice line. A repeated product on the same invoice remains several fact rows if the source contains several line identifiers.

11 Grain keys and measurable quantities

Grain is the exact meaning of one row. It must be defined before choosing the key or adding measurements. Changing grain without changing the key can duplicate amounts or overwrite valid records.

Table	Grain	Key
dim_produit	One product identifier	Product_Id
dim_client	One customer/company member	Company_Id
dim_fournisseur	One supplier member	Fournisseur_Id
dim_paiement	One payment mode	PaymentMode_Id
dim_temps	One calendar date	DateKey
fait_stock_actuel	One product in the current source snapshot	Product_Id
fait_achats	One eligible source invoice line	Id from DocumentItem

A primary key, PK, uniquely identifies a row and cannot be NULL. A foreign key, FK, requires a referenced dimension member to exist, unless that FK column permits NULL. This model keeps existing business identifiers as dimension keys; it does not introduce separate surrogate identifiers.

A measure is a quantitative or calculated value used in analysis. Purchased quantity and net purchase amount are measures. Product family and supplier name are analysis attributes. An invoice identifier is a reference, not a value to sum.

Additivity describes which sums make business sense. Purchase amounts can be summed across eligible lines. Quantities should be grouped by compatible products or units. Current stock quantities must not be summed across repeated snapshots. Alert thresholds and unit prices are not additive measures.

NULL means missing or not known, not zero and not an empty string. A missing stock threshold must not automatically become zero. The supplier key -1 is a real unknown-member row, whereas a NULL stock DateKey means there is no proven calendar reference.

12 Product dimension Data Flow

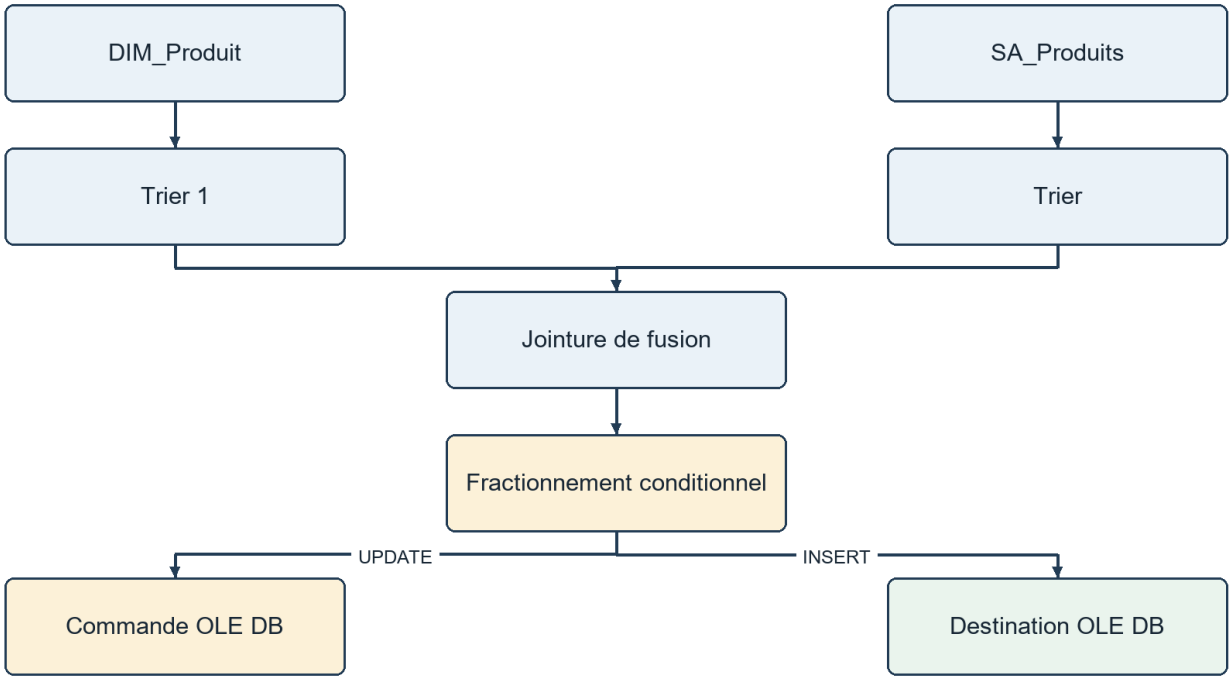


Figure 10. Actual component topology reconstructed from alim_dim_produit. Original French component names are retained; this is a documentation diagram, not a screenshot.

The source SA_Produits executes usp_source_dim_produit and declares the result columns explicitly: Product_Id, Libelle, Famille and Marque. DIM_Produit supplies existing warehouse members. Both inputs are sorted by the joining product identifier before the Merge Join.

Sort, called Trier in the designer, orders rows by the merge key. Merge Join, called Jointure de fusion, combines the sorted source and warehouse rows. The Conditional Split then selects new products for insertion and changed products for an OLE DB Command update. Equal members need no write.

The resulting dimension contains 6842 product members. Libelle is the descriptive name; Famille is the family; Marque is the brand. Their values can come from several sources, which is why a separate matching procedure is needed before the visible comparison and loading steps.

The physical schema permits a nullable product label, but the final quality checks require a nonempty label. If a reliable description cannot be reconstructed, the procedure returns an explicit label such as PRODUIT INCONNU followed by the product identifier.

13 How product descriptions are reconstructed

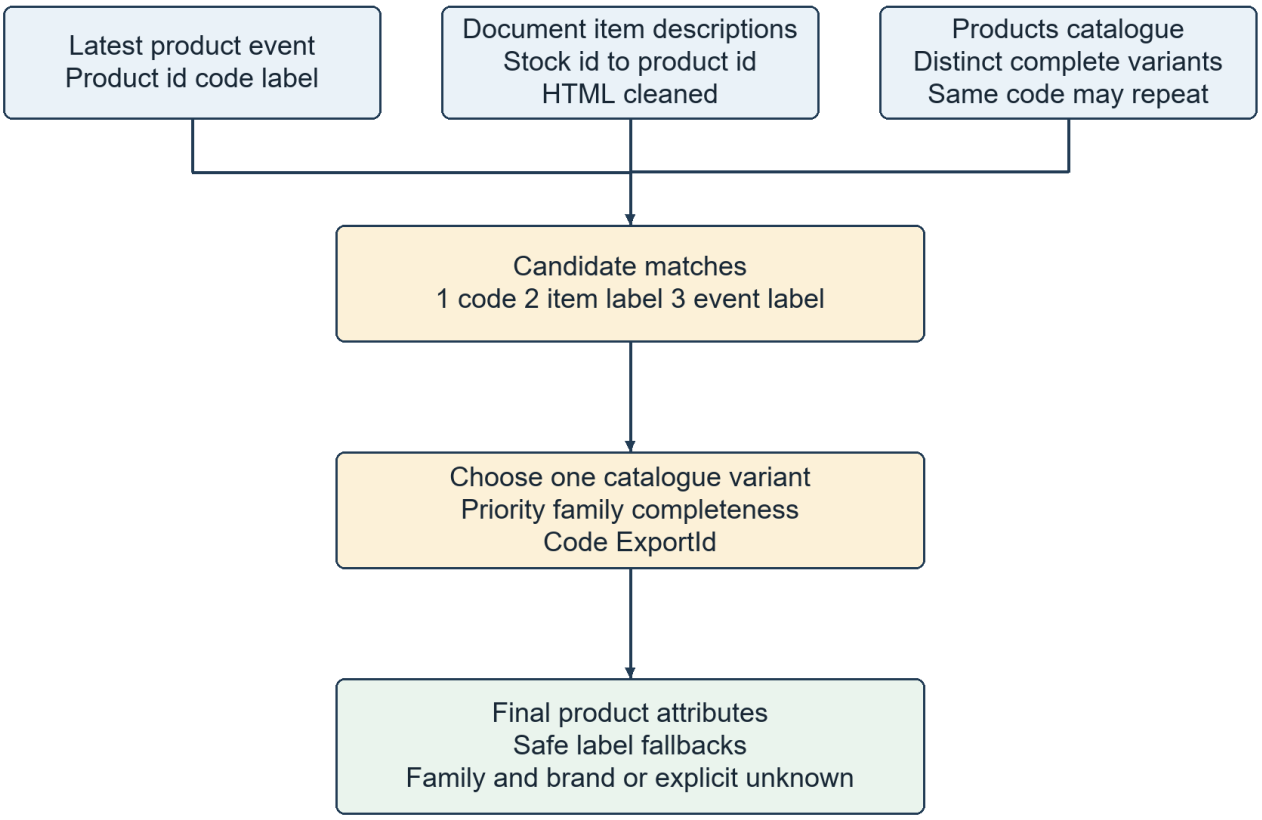


Figure 11. The source procedure builds evidence and chooses one catalogue variant. It does not globally assume that a product code is unique.

The procedure keeps the latest product event by event identifier and extracts product id, code and label from JSON. JSON is structured text made of named values. It also links document item descriptions to products through ProductStocks, cleans HTML and builds alternative label evidence.

Catalogue records are deduplicated by code, label, family and brand together. A technical ExportId distinguishes the retained variants. This fixes the earlier assumption that one code always represented one reliable catalogue row.

Candidate priority is code agreement first, document-line label agreement second and event-label agreement third. An ambiguous repeated code must also agree with the event label. Candidate ordering then prefers a populated family and uses code and ExportId to choose one result.

An exact binary test rejects catalogue labels containing the replacement character. Label fallback is then document-line description, event label, and finally an explicit unknown label. Family defaults to NON RENSEIGNE. Brand uses the catalogue, a known-brand prefix inferred from the chosen label, or NON RENSEIGNE. Missing letters are not invented.

14 Client dimension Data Flow

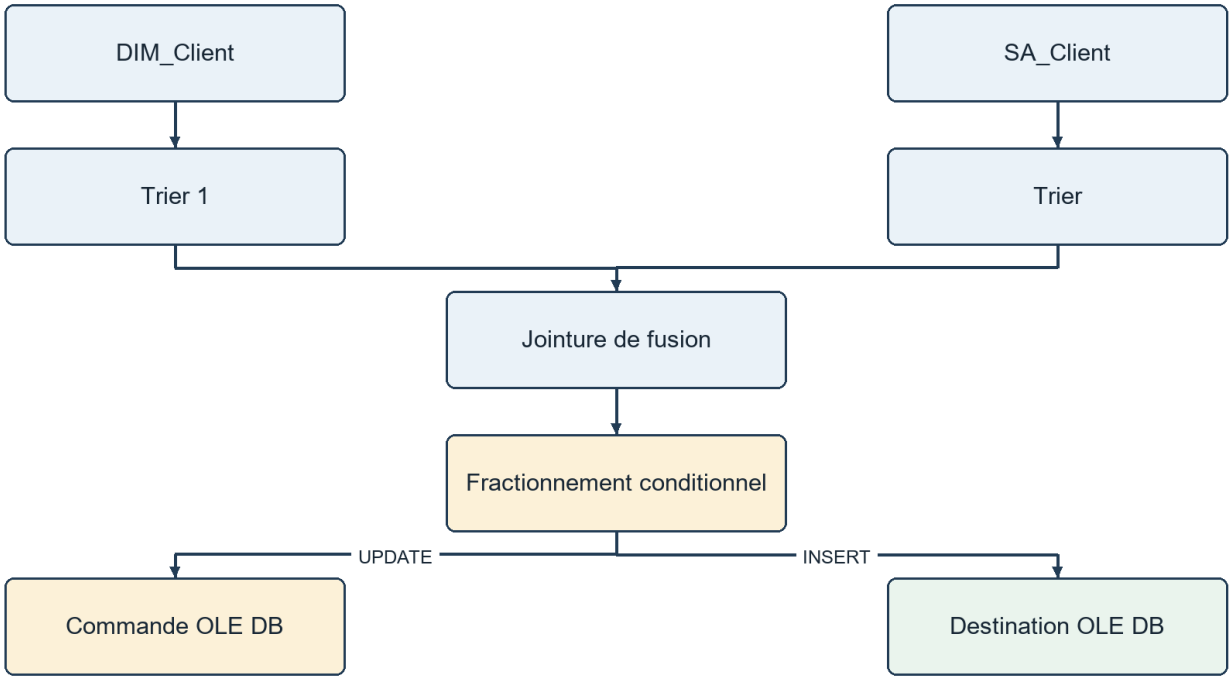


Figure 12. The original client comparison pattern uses two sorted inputs, Merge Join, Conditional Split and separate insert and update destinations.

Candidate identifiers come from eligible validated sales documents and their transactions, explicit special members, and company/customer-export code matches. Related-document and product-line conditions help define the source client population. These rules support the dimension even though a sales fact is outside the final scope.

The name is taken from the repaired customer label or company name. An em dash can separate a descriptive prefix from the name. TypeClient classifies known labels as PARTICULIER, B2B - LABORATOIRE, COMPTOIR_ANONYME or NON RENSEIGNE.

Company_Id 2 is Client comptoir (anonyme), a walk-in anonymous customer. It must not be counted as an identified person. The unknown member -1 and anonymous member 2 have EstClientIdentifie = 0; other retained identifiers have 1. The dimension has 82 members, not 82 identified people.

The corrected comparisons update a name or classification even when the existing destination value is NULL. Changes replace existing attributes, following Slowly Changing Dimension type 1, abbreviated SCD type 1. No history row is created.

15 Supplier dimension Data Flow

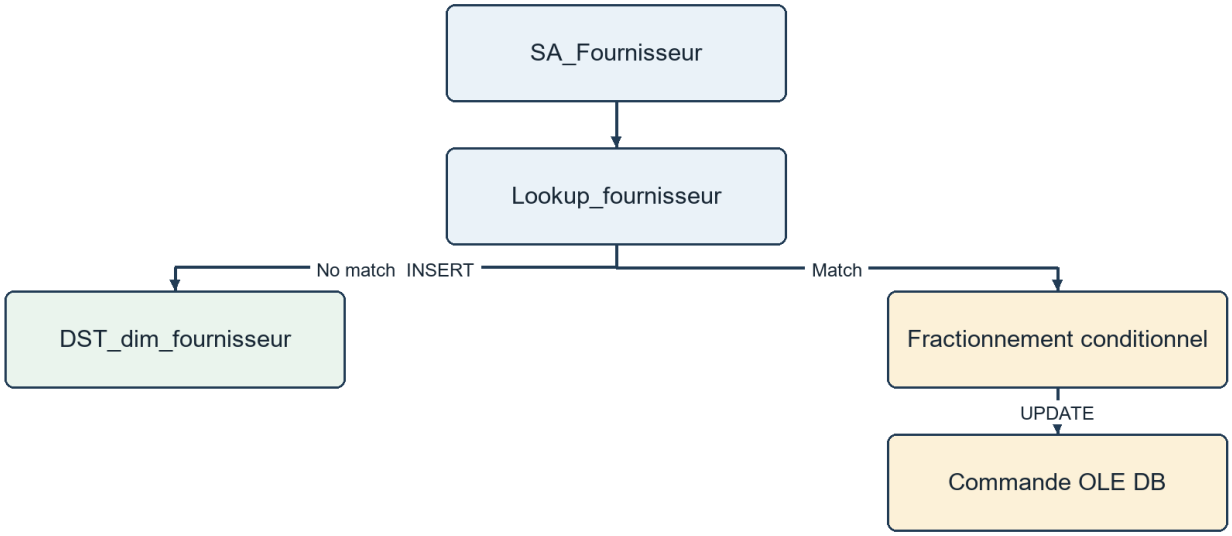


Figure 13. Actual supplier topology. A Lookup no-match path inserts a member; matched members go through change detection before UPDATE.

Supplier candidates include companies marked CompanyType_Id = 2 and identifiers required by validated purchase invoices or supplier credit notes. Including credit-note suppliers in the dimension does not mean credit-note facts are loaded.

The source combines CompaniesList and the supplier export by business code. It removes the supplier label's numeric list prefix when applicable and repairs known accent sequences. It falls back to the company name or an explicit supplier-unknown label when necessary.

Lookup, called Recherche in some diagrams, searches the reference table for a matching key. If the key is absent, the destination inserts it. If present, the Conditional Split compares names and routes only a change to the OLE DB Command. The NULL-safe expression prevents a missing old name from stopping the flow.

There are 116 supplier members, including the -1 NON RENSEIGNE member. Thirteen eligible purchase lines have an unassigned source supplier and reference -1. Empty identifiers are normalized to NULL before conversion, preventing SQL from accidentally converting an empty string into key 0.

16 Payment mode dimension Data Flow

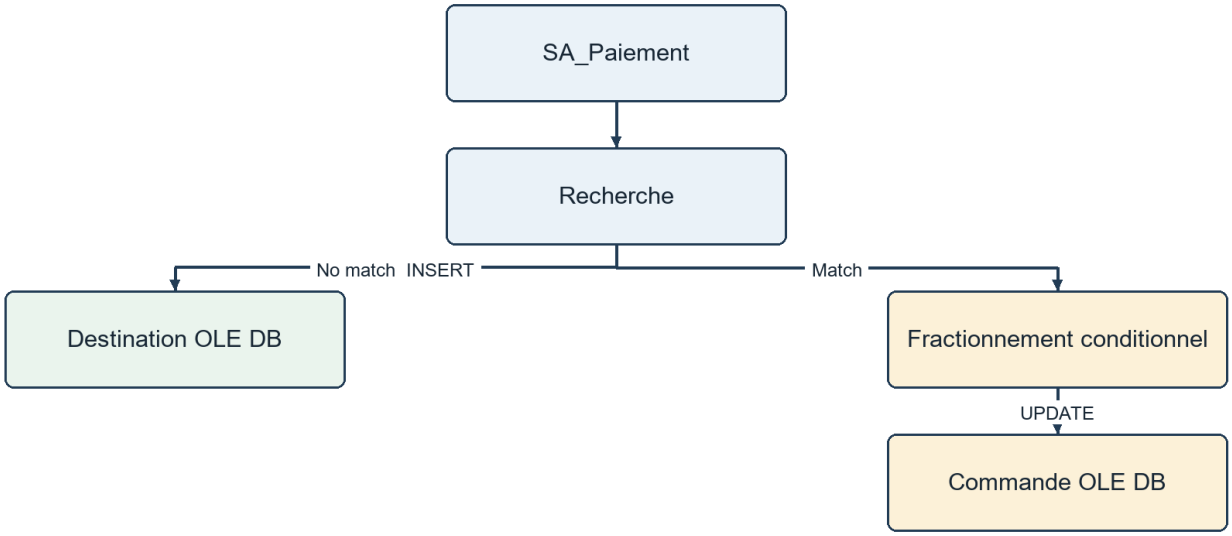


Figure 14. The payment-mode diagram retains Amal's Lookup, no-match INSERT and matched-row Conditional Split UPDATE pattern.

The source reads distinct payment modes from PaymentsList_SA and translates known application labels into readable names. The resulting identifiers are 1, 5, 7, 11, 21 and -1, covering cheque, cash, bank card, deferred payment, tokens and an unknown member.

The mode name is PaymentModeName. It describes how payment occurs. It is not a monetary measure and does not mean the invoice has already been paid. The mode named credit or deferred payment is also different from a supplier credit note, which is a document reversing part of a purchase.

This version retains all six payment-mode dimension members but does not create a payment fact. Neither current stock nor purchase invoice lines require a payment-mode FK. Joining every payment directly to every invoice line could multiply amounts, so the purchase source deliberately avoids that join.

As in the supplier flow, new identifiers are inserted and changed labels are updated. Existing NULL values are compared explicitly. The regression tests intentionally change a payment label and verify that the source value is restored.

17 Time dimension and the business calendar

The Master task Calendrier runs the calendar SQL directly. It inserts missing dates from 30 September 2024 through 29 September 2026 inclusive, giving 730 rows. It does not delete existing dates or reload them unnecessarily.

Field	Meaning	Illustration
DateKey	Integer date key in YYYYMMDD format	20241113
Date	Actual SQL date	13 November 2024
Annee	Calendar year	2024
Trimestre	Calendar quarter from 1 to 4	4
Mois	Month from 1 to 12	11
SemaineISO	ISO week number	Based on ISO week rules, not a simple count from 1 January
EstFerie	Holiday indicator	1 for the configured holiday set, otherwise 0

DateKey is calculated as year times 10000 plus month times 100 plus day. Purchase DateKey uses the invoice date. The SQL source parses the original month/day/year representation explicitly with conversion style 101; it does not rely on an ambiguous machine locale.

The inherited holiday logic uses the French holiday set: 1 January, 1 and 8 May, 14 July, 15 August, 1 and 11 November, 25 December, Easter Monday, Ascension and Whit Monday. Easter is calculated using the Gregorian algorithm. The business country must confirm this inherited choice; this is not a Tunisian holiday implementation.

The calendar extends beyond the real purchase dates and does not generate synthetic purchase records. There is no fabricated capture date for stock. A future snapshot history would require a reliable capture date and a different product-by-date fact key.

What the calendar check proves

The validation checks 730 rows and verifies DateKey, year, month, quarter and ISO week against the stored SQL date. It verifies consistency with the implemented calendar rules, not the appropriateness of the holiday country for every business.

18 Shared transformation rules

fn_reparer_accents replaces a limited list of known mojibake sequences under a binary collation. fn_texte_html removes HTML tags by replacing them with spaces, decodes selected entities such as ampersands and nonbreaking spaces, and trims/collapses extra spaces. These functions are applied in the relevant source logic, not as an undocumented global rewrite of every raw column.

Collation is the rule SQL Server uses to compare and sort text. CI_AI comparisons ignore case and accents for matching labels. Binary comparisons inspect exact character values when identifying corruption. The binary check matters because the replacement character can behave unexpectedly under a linguistic collation.

New value	Existing value	Changed
NULL	NULL	No
A known value	NULL	Yes
NULL	A known value	Yes
The same non-NULL value	The same value	No
A different non-NULL value	An old value	Yes

A normal new_value != old_value expression can return NULL if either side is NULL. A Conditional Split needs a definite Boolean result. The implemented ternary expression first checks missingness, and only compares values when both are non-NULL.

```
ISNULL(NewValue) ? !ISNULL(OldValue) :
    (ISNULL(OldValue) ? TRUE : NewValue != OldValue)
```

fn_nombre_fait removes ordinary and selected nonbreaking spaces, changes a decimal comma to a dot and uses TRY_CONVERT(decimal(18,4)). TRY_CONVERT returns NULL instead of raising a conversion error. The mandatory-value guards then decide whether that missing result is acceptable or must stop the load. A failed numeric conversion is not silently replaced with zero.

19 Prepare the facts before loading

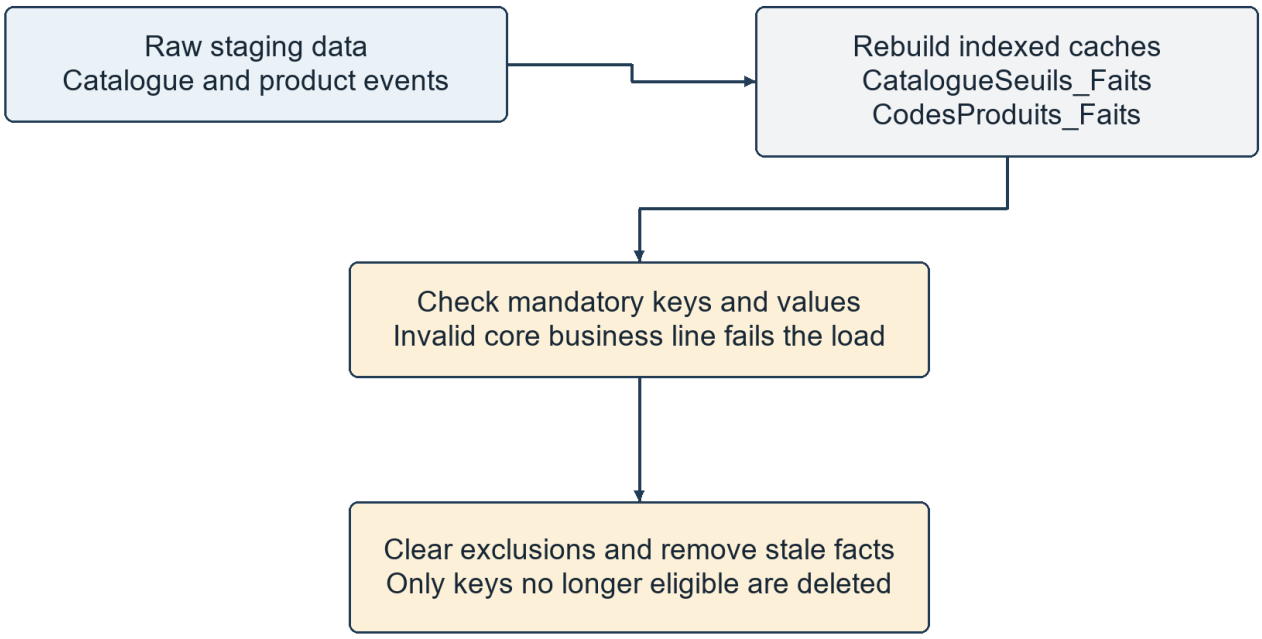


Figure 15. usp_preparer_faits is called automatically by the preparation SQL task after the dimensions. It prepares inputs and removes ineligible keys; it does not insert the final warehouse facts.

CatalogueSeuils_Faits caches catalogue code, repaired label and numeric threshold. CodesProduits_Faits caches the latest product-event code with Product_Id as a primary key. Indexes make these repeated lookups inexpensive. Both caches are rebuilt from current staging before the fact Data Flows run.

Caching was introduced because repeated repairs and repeated JSON extraction inside joined queries made stock validation slow. In the recorded performance check, reading the full stock candidate aggregate after preparation took about 250 milliseconds. This is a measured local query result, not a promised runtime for the whole pipeline.

The guards reject stock records with an invalid product key or quantity. For validated purchase lines they require valid line/document identifiers and dates. A commercial product line also requires quantity, amount and a calculable net amount. A no-product line with a nonzero amount fails instead of disappearing as a comment.

The procedure clears Rejets_Faits, deletes stock facts whose product key is absent from current candidates, and deletes purchase facts whose line is no longer a validated commercial candidate. This deletion is necessary for source status changes; an upsert alone would leave a previously valid but now-draft invoice in the warehouse.

20 Current stock Data Flow

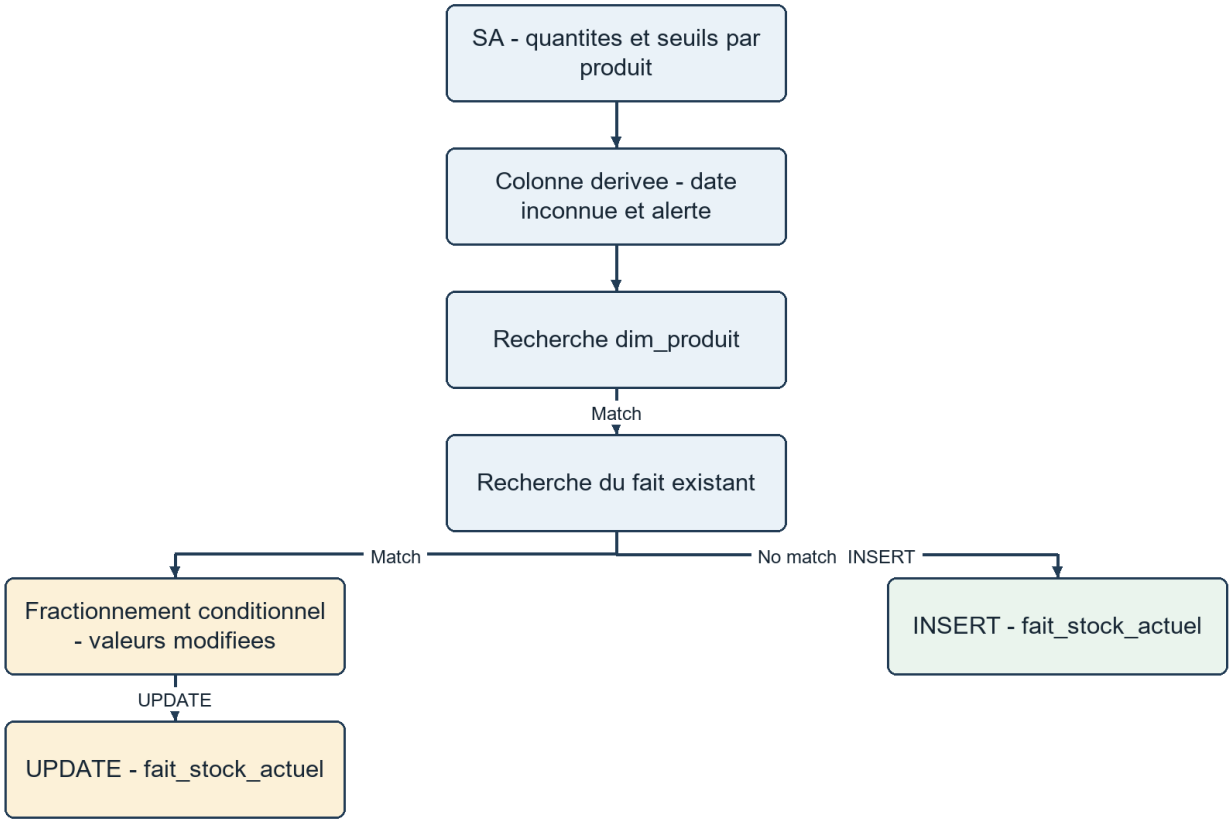


Figure 16. Actual alim_fait_stock_actuel topology. The diagram shows source, derived columns, product Lookup, existing-fact Lookup and separate native INSERT and UPDATE paths.

v_source_stock groups source ProductStocks rows by typed Product_Id and sums their quantities. If a stock quantity cannot be converted, the product's derived quantity becomes NULL and preparation fails explicitly. Negative balances are retained rather than changed to zero.

The source retrieves alert thresholds by the latest product-event code first, then by the conformed product label as a fallback. Threshold groups return a value only when the non-NULL candidates agree. An absent or conflicting threshold remains NULL. The view uses dim_produit as a label reference after that dimension has loaded.

The Derived Column transformation sets DateKey to NULL and calculates EstSousSeuil. Lookup dim_produit checks the product reference. Lookup of the existing fact separates a new product from a current stock row. Changed measures are updated; an identical row needs no write. A missing required dimension reference fails the flow.

21 Understand stock measures and unknown dates

Column	Source or calculation	Business meaning
Product_Id	Typed ProductStocks.Product_Id	The current product key, also the fact PK
DateKey	NULL by design	The capture date is not established by the export
QuantiteActuelle	Sum of ProductStocks.Quantity by product	Current source balance
QuantiteAlerte	Consistent catalogue threshold using code or label matching	Reorder comparison threshold; may be unknown
EstSousSeuil	Quantity < threshold; NULL if threshold is NULL	1 means below threshold, 0 means not below

Worked example

Suppose Product A has a balance of 8 and an alert threshold of 10. Its alert indicator is 1 because 8 is less than 10. A balance of exactly 10 produces 0. If the threshold is missing, the indicator is NULL; it is not evidence that the product is safe or below threshold.

The delivered snapshot has 6842 products and total source quantity 18800. There are 4449 products with known thresholds and 2195 below their known threshold. The remaining 2393 thresholds are unknown. These figures must be interpreted with product-unit compatibility in mind; the total quantity is a source reconciliation, not a universal business unit.

The date of the last sale, last stock movement or current ETL execution does not prove the date of the exported stock snapshot. DateKey therefore remains NULL. This is an explicit limitation, not a failed date conversion.

To keep several captures in the future, use a reliable capture date and change the grain and key to product plus capture date. The current Product_Id primary key intentionally overwrites the same product’s current balance instead of retaining daily history.

Alert thresholds are control values, not values to sum in a dashboard. Compare each product’s balance with its own threshold. The stock flag is calculated at the product grain and can be counted to answer how many products need attention.

22 Construct purchase candidates from staging

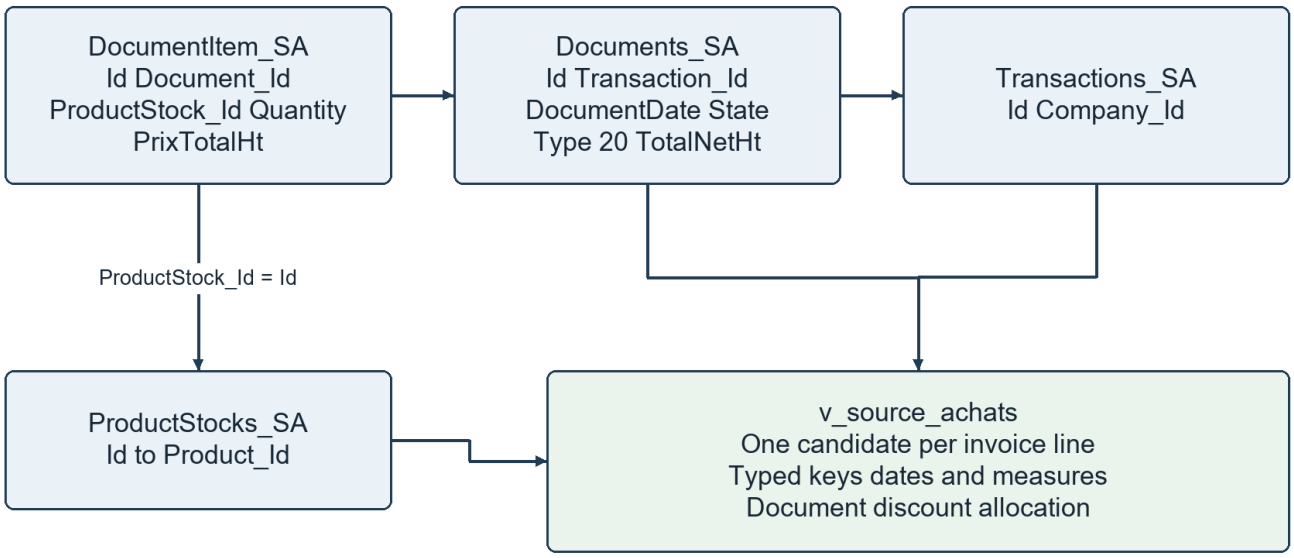


Figure 17. The purchase source joins four staging subjects. Document line identifiers preserve the fact grain; payment rows are not part of this join.

DocumentItem supplies line Id, Document_Id, ProductStock_Id, Quantity and PrixTotalHt. Documents supplies the invoice date, state, type, transaction reference and TotalNetHt. ProductStocks translates the stock identifier to Product_Id. Transactions supplies the supplier/company identifier.

The source view filters to DocumentType_Id 20, the supplier invoice type in this data. IsValidated is true only for State 1. The view still presents draft invoice lines to the Data Flow so that the Conditional Split can document their exclusion. Supplier credit notes, type 23, are outside this fact scope.

Before numeric conversion, empty identifier strings become NULL using NULLIF after trimming. This prevents the empty-string-to-zero conversion that originally created unmatched supplier key 0. Unassigned suppliers become the existing member -1.

The source uses joins and window calculations to prepare net line amounts. A window function calculates values within an invoice while retaining each individual line row. It does not collapse an invoice to one fact record. All joining and allocation logic is installed in the documented SQL view, read by the native OLE DB Source.

23 Purchase selection and dimension lookups

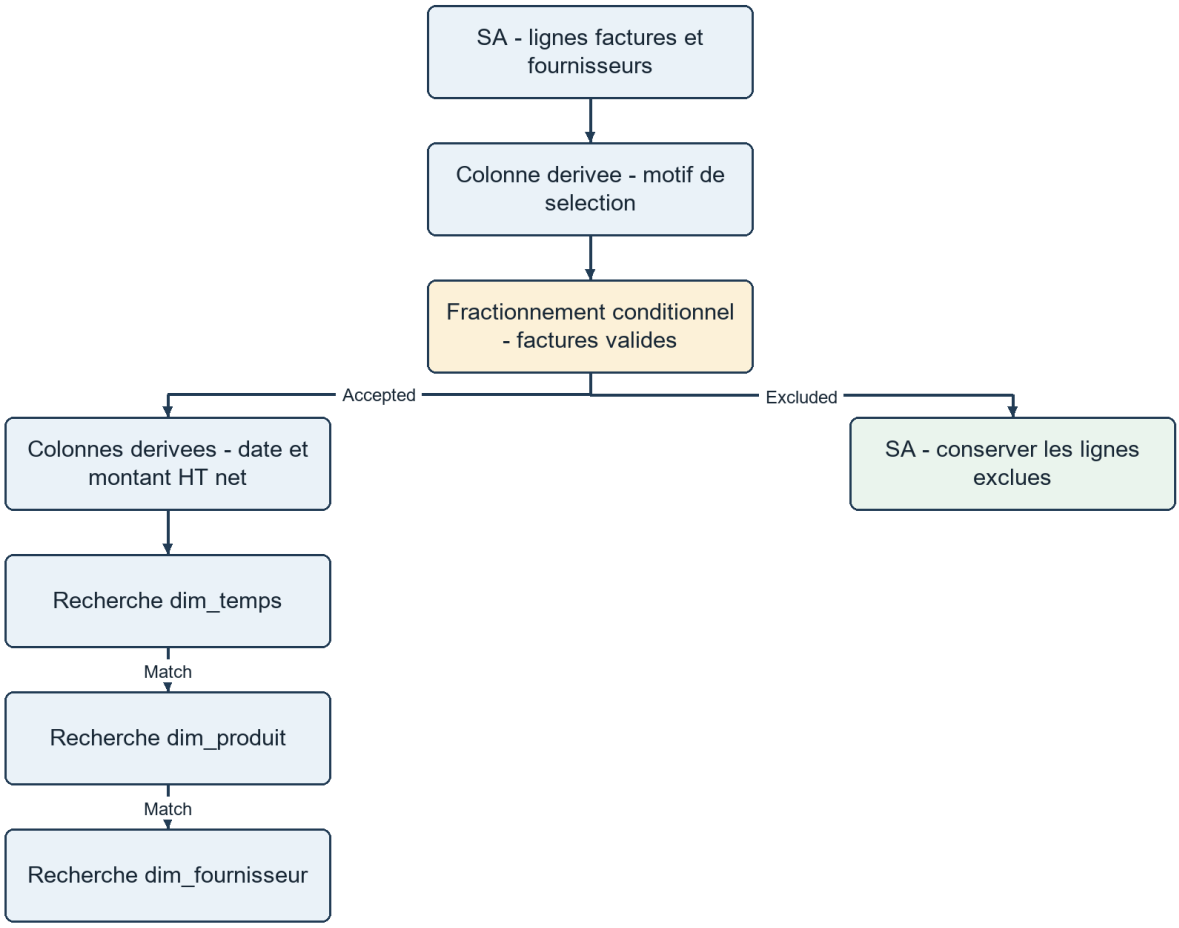


Figure 18. The first part of the actual purchase Data Flow. The existing-fact upsert that follows is shown in the next figure. This split presentation keeps the full flow readable.

The first Derived Column assigns the fact name and a selection reason. BROUILLON means draft. SANS_PRODUIT identifies no-product candidates. ACCEPTE is the accepted commercial route. The Conditional Split sends excluded rows to Rejets_Faits in staging, retaining source line Id and reason.

Accepted rows receive DateKey, QuantiteAchetee and MontantAchat_HT through visible Derived Column expressions. Three Lookups verify date, product and supplier keys. Their required-reference behavior is fail on no match; a broken dimension reference is not silently ignored.

The measured source contains 7027 type-20 candidate lines. Of these, 6964 commercial lines on validated invoices are loaded, 49 draft lines and 14 valid-invoice comment lines are logged as exclusions. This accounts for every candidate row in this dataset.

24 Native insertion and update decisions

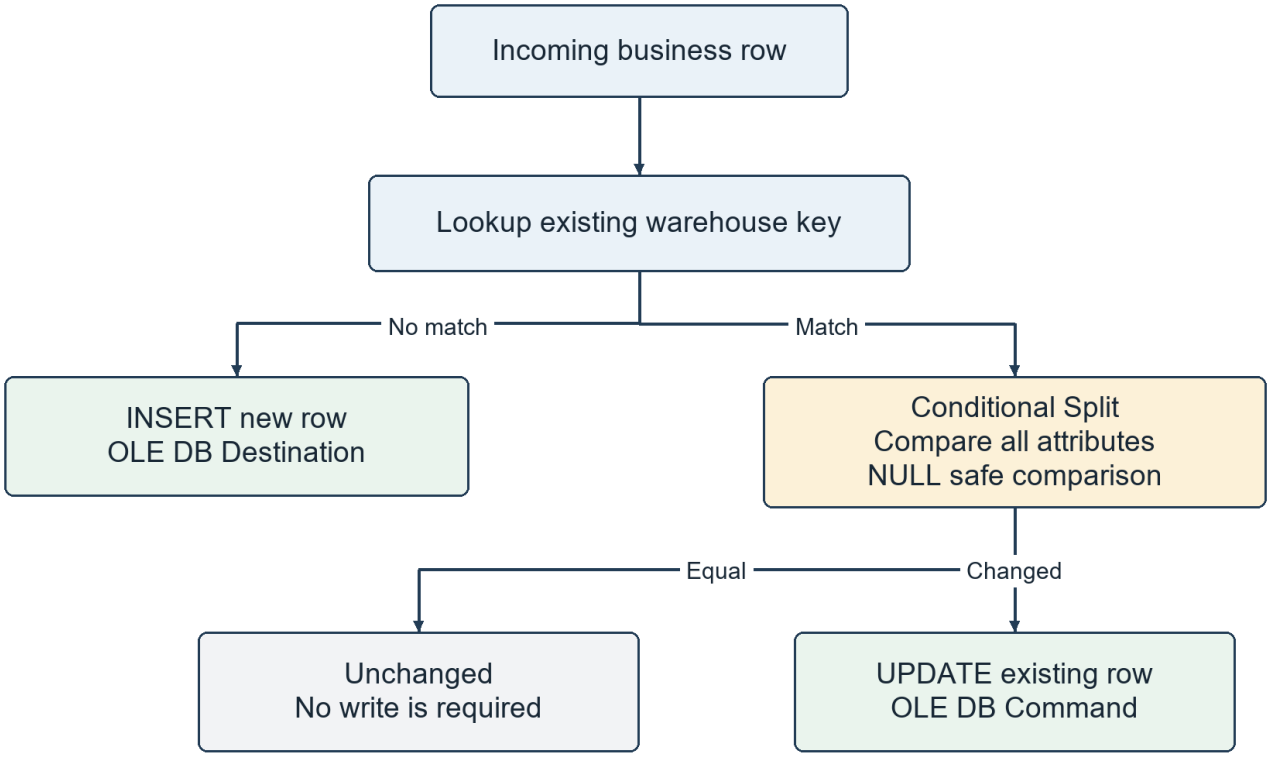


Figure 19. The shared existing-key pattern used by both fact Data Flows. This logical diagram also makes the intentionally unwritten unchanged route explicit.

Upsert means insert a missing key or update an existing one. Lookup of the warehouse fact key is separate from the earlier dimension key checks. Product_Id identifies a stock fact; source line Id identifies a purchase fact.

The no-match path uses an OLE DB Destination for new rows. The match path enters a Conditional Split that compares every stored attribute or measure using NULL-safe expressions. Changed rows reach an OLE DB Command with parameterized UPDATE SQL. Equal rows require no warehouse write.

The stock comparison covers DateKey, quantity, threshold and alert. The purchase comparison covers document, date, product, supplier, quantity and amount. Destination fast load uses CHECK_CONSTRAINTS so the SQL foreign keys remain enforced and trusted.

An OLE DB Command runs a parameterized SQL command for an input row. This approach follows the original supplier/payment style and is easy to inspect in the designer. It is not a claim that row-by-row updates are the most scalable design for every production warehouse.

25 Calculate net purchase amounts correctly

PrixTotalHt is the line HT amount after its own line discounts. Some invoices also have a document-level discount. Summing the lines without applying that global adjustment would overstate net purchases. In this dataset, 53 validated invoices differ between their line HT sum and header TotalNetHt by more than 0.01.

Proportional allocation rule

For an invoice, let S be the sum of commercial line HT amounts and N be the header TotalNetHt. Each line receives its share of N in proportion to its original line amount. The proportion is represented to eight decimal places, and each line amount is rounded to four decimal places. The residual needed to reach exactly N is assigned to the commercial line with the greatest source Id.

Fictional line	Base HT	Share of invoice net HT	Result
Line 101	120.0000	120 / 200 of 180	108.0000
Line 102	80.0000	80 / 200 of 180	72.0000
Invoice	200.0000	Net header 180.0000	180.0000

The allocated document discount is the original line HT minus its final net share. The visible SSIS Derived Column then calculates the warehouse amount by subtracting that share of document discount. The discount is not applied twice.

```
MontantAchat_HT = (DT_NUMERIC,18,4)
    ([TotalHt] - [RemiseDocument_HT])
```

If S and N are both zero, original line values are preserved. If a nonzero net header cannot be allocated from the available line amounts, the calculated amount is missing and a validated commercial line fails the mandatory-value guard. It is not silently accepted with a fabricated amount.

26 Rounding and purchase field lineage

Rounding residual means the small difference between a required total and the sum of separately rounded parts. For example, three base lines of 1.0000 with invoice net 1.0000 produce three provisional shares of 0.3333. Their sum is 0.9999. Adding 0.0001 to the line with the greatest source Id gives 0.3334, 0.3333 and 0.3333, totaling exactly 1.0000.

Warehouse field	Origin or rule
Id	DocumentItem.Id converted to bigint; one row per source line
Document_Id	Documents.Id; invoice reference retained in the fact
DateKey	YYYYMMDD calculated from the explicitly parsed invoice date
Product_Id	DocumentItem.ProductStock_Id to ProductStocks.Id to Product_Id
Fournisseur_Id	Documents.Transaction_Id to Transactions.Id to Company_Id; -1 if unassigned
QuantiteAchetee	Typed DocumentItem.Quantity
MontantAchat_HT	Line PrixTotalHt minus allocated global discount, including rounding correction

The warehouse holds 6964 eligible lines belonging to 880 invoices. The base line HT sum is 964358.5720; net purchases after document adjustments are 958947.7230. The difference is 5410.8490. The final assertions reconcile every invoice individually, so agreement is stronger than a coincidentally matching grand total.

These amounts are net of discounts but exclude supplier credit notes. They are therefore not a net-after-returns purchase measure. No currency unit has been invented in this guide. Unit prices, TTC tax-inclusive amounts and cash settlements are not additional measures in this fact.

27 What happens when the load runs again

Idempotence means the same input produces the same final business tables after another execution. Here the source staging snapshot is refreshed, the calendar inserts only missing dates, dimensions follow SCD type 1, and existing facts are compared before an update.

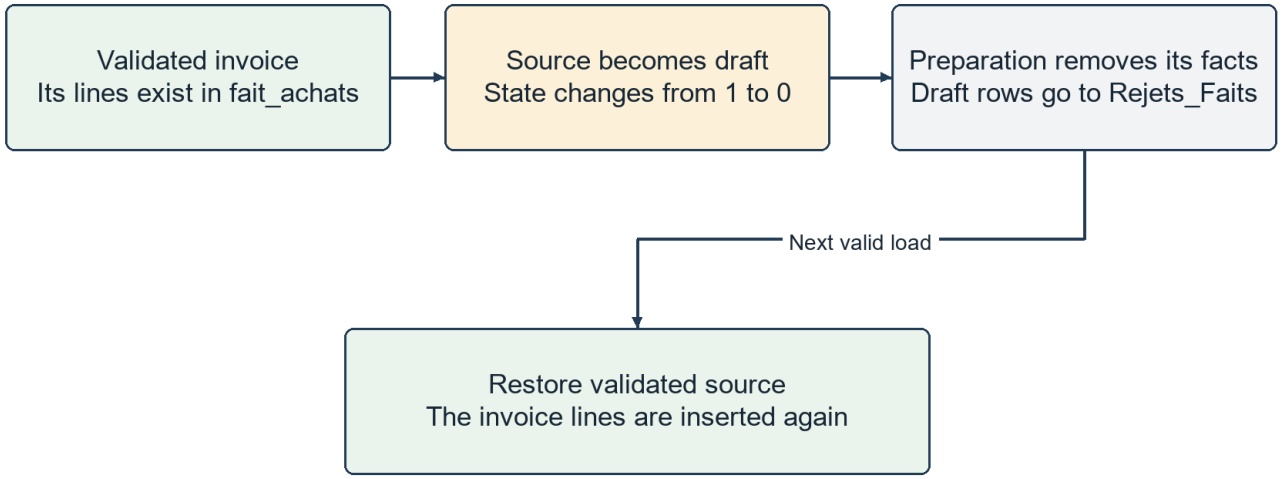


Figure 20. A source change can remove a fact as well as update its measures. The regression test reverses the change and verifies exact restoration.

An upsert by itself cannot remove a line that has become ineligible. The preparation procedure therefore deletes stale keys before the native fact flows. When a source invoice changes from validated to draft, all its purchase fact lines are removed. Its candidate rows then follow the draft exclusion route. If it becomes validated again, its commercial line identifiers follow INSERT.

For stock, a product no longer present in the current stock candidates is removed from the current snapshot fact. Dimensions are different: the SCD type-1 flows do not automatically purge a dimension member when it disappears from source. This keeps the final implementation’s deletion policy explicit.

The tests compare hashes of every final dimension and fact before and after the second full Master execution. They also change destination values and a source invoice state to demonstrate repair, deletion and reinsertion rather than checking only a first successful load.

28 Validation and the meaning of 67 checks

The recorded suite has 67 successful checks and zero failures. These are script assertions and execution checks, not 67 unrelated unit-test functions. Several checks verify entire tables. Appendix C lists every recorded check by name.

Check family	Count	Evidence
Native package execution	6	Two full Masters and four focused ETL executions
Staging source row counts	17	Each raw staging table equals its manifest count
Staging cell hashes	17	All parsed source cells and duplicate rows reconcile
Second-run table stability	7	Same keys and attributes in all seven warehouse tables
Dimension repair and resulting table stability	7	Three NULL labels and a changed payment name are repaired
Known-threshold fixture prerequisite	1	The NULL stock test targets an actually known threshold
Fact UPDATE repair	2	Stock and purchase hashes return to baseline
Reclassified invoice removal	1	No fact line remains for the temporary draft invoice
Restoration after source fixture	7	All seven tables match the original baseline again
SQL reconciliation and FK checks	1	Both fact directions, headers, calendar and trusted keys
Warehouse table population	1	The report records five dimensions and two facts

SHA256 is a cryptographic hash used here as a compact fingerprint, not encryption. Each cell is represented with its UTF16 byte length followed by its bytes. Each row is hashed; row hashes are sorted with duplicate occurrences retained; the complete sorted list is hashed again. This multiset comparison ignores SQL row order while preserving duplicate counts.

CSV empty cells and SQL NULL are treated equivalently in this cell check. Original file byte hashes are recorded separately. Hash agreement supports transfer fidelity under that defined normalization; it does not certify that an absent product description is factually known.

29 Final SQL assertions and recorded results

The final Verification task requires exactly the seven intended warehouse tables. It checks calendar size and attributes, special dimension members, required descriptions, six payment modes, and enabled trusted foreign keys. A trusted FK has been validated against existing rows; it is more than a constraint name stored in metadata.

For each fact, the SQL uses differences in both directions: source candidates minus loaded facts and loaded facts minus source candidates. Keys and measures must agree, not just row counts. The purchase check also groups loaded facts by invoice and requires the sum of MontantAchat_HT to equal source TotalNetHt exactly.

Warehouse table	Loaded rows
dim_produit	6,842
dim_client	82
dim_fournisseur	116
dim_paiement	6
dim_temps	730
fait_stock_actuel	6,842
fait_achats	6,964

The stock total is 18800.0000. There are 4449 known thresholds, 2393 unknown thresholds and 2195 products below their known threshold. Purchase net HT is 958947.7230 over 880 invoices. The exclusion log ends with 49 draft lines and 14 no-product comment lines. Five foreign keys are enabled and trusted.

The measured product dimension still contains 2182 explicit unknown labels, 2543 unknown families and 3055 unknown brands. These figures describe missing or unmatched source descriptions. Integration success does not turn an unknown source value into verified business knowledge.

The evidence files are verification-results.json, implementation-summary.json, source-manifest.json, build.log, master.log, master-rerun.log, dimension-null-and-update.log, facts-update.log, facts-source-reclassification.log, facts-source-restored.log and sql-validation.log in reports. FINAL-REPORT.md is the human-readable summary.

30 Run the project on the current machine

Open and execute in Visual Studio

Open HealthRetailers_SSIS.sln from the HealthRetailers_DW_amal folder. If Visual Studio reports that package files changed outside the editor, reload them. Open Master.dtsx to see staging, calendar, warehouse loading and verification. Execute Master as the entry package. The required runtime has already been installed on this machine.

To inspect transformations, open ETL_SA_DW.dtsx and double-click a task named alim_dim_client, alim_dim_produit, alim_dim_fournisseur, alim_dim_paiement, alim_fait_stock_actuel or alim_fait_achats. The first four are the retained dimension diagrams; the final two are the added fact flows.

View the SQL data

In SSMS, connect to localhost with Windows Authentication. Expand Databases, then SA_Health_Dimensions for staging or DW_Health_Dimensions for the warehouse. Refresh Object Explorer if a newly created table is not yet visible. SSMS can be closed while a package runs.

```
USE DW_Health_Dimensions;
SELECT COUNT(*) FROM dbo.fait_stock_actuel;
SELECT COUNT(*) FROM dbo.fait_achats;
```

Run through the supplied scripts

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File scripts\setup.ps1
powershell.exe -NoProfile -ExecutionPolicy Bypass -File scripts\verify.ps1
```

Run the commands from the HealthRetailers_DW_amal project folder. setup.ps1 is for setup or intentional reconfiguration; it is not required before every Master run. verify.ps1 runs the regression suite and temporarily modifies controlled fixtures before restoring them. Avoid concurrent manual package runs while that verification is executing. The normal operating entry point is Master, not the fixture tests.

31 Configure or move the project

settings.example.json records the default server and database names. Copy it to settings.json if a deliberate configuration change is needed, then run setup.ps1. configure.py reads the local settings, creates transport copies, replaces connection paths and changes the project target to SQL Server 2022.

```
{
  "Server": "localhost",
  "StagingDatabase": "SA_Health_Dimensions",
  "WarehouseDatabase": "DW_Health_Dimensions"
}
```

There is no implemented SSIS project parameter named DossierDonnees. Paths are configured by the supplied setup process. Database connection managers and the added fact package connections use integrated Windows authentication.

A solution file, .sln, groups the project for Visual Studio. The .dtproj file defines the SSIS project and target version. A .dtsx file stores a package. A .conmgr file stores a connection manager. The build produces HealthRetailers_SSIS.ispac under HealthRetailers_SSIS/bin/Development, a compiled project deployment artifact used by DTExec.

The raw business data and transport copies are excluded from Git. A fresh clone on another machine therefore also needs the source files listed in scripts/sources.json under data/original. The current machine already has them. configure.py can use the adjacent original repository as a local fallback; that fallback is not an internet download service.

The required environment is SQL Server 2022, full SSIS 2022 runtime, Visual Studio 2022 with the SSIS Projects extension, the Microsoft OLE DB SQL Server provider, Python and sqlcmd. The C# authoring helper uses installed SSIS assemblies during setup; the saved runtime flows use native SSIS components.

32 Analyse the warehouse in SSMS

sql/07_analyses.sql contains read-only examples. To run the file directly in SSMS, enable SQLCMD Mode and define the variable with :setvar WarehouseDatabase DW_Health_Dimensions. The queries below can also run directly in the warehouse database. Analysis should respect the grain and unknown values. Product family or brand joins come from dim_produit; month and supplier joins come from dim_temps and dim_fournisseur.

Find products that need attention

```
SELECT p.Product_Id, p.Libelle,
       f.QuantiteActuelle, f.QuantiteAlerte
FROM dbo.fait_stock_actuel f
JOIN dbo.dim_produit p ON p.Product_Id = f.Product_Id
WHERE f.EstSousSeuil = 1;
```

This returns products below a known threshold. Products with unknown thresholds are not included by that predicate; report their count separately instead of assuming they need no action.

Compare net purchases by month and supplier

```
SELECT t.Annee, t.Mois, s.Nom,
       SUM(f.MontantAchat_HT) AS MontantAchat_HT
FROM dbo.fait_achats f
JOIN dbo.dim_temps t ON t.DateKey = f.DateKey
JOIN dbo.dim_fournisseur s
  ON s.Fournisseur_Id = f.Fournisseur_Id
GROUP BY t.Annee, t.Mois, s.Nom;
```

This sums invoice-line net amounts after the document discount allocation. It excludes supplier credit notes by design. Quantities are better analysed by product or compatible units. Do not join a one-to-many payment table into this result without a separate allocation design.

33 Diagnose a failed execution

Symptom	Likely interpretation	What to inspect
Database connection fails	Wrong server, permissions or provider setup.	settings files, connection managers, SQL Server availability
Flat File Source fails	Path, encoding, delimiter, quoting or width issue.	Original/transport files, source manifest, flat-file manager
Preparation reports invalid core values	A mandatory identifier, date, quantity or net amount is invalid.	v_source_stock, v_source_achats and the original raw rows
Dimension Lookup reports no match	A required warehouse reference is absent.	Candidate key and its corresponding dimension member
Conditional Split cannot evaluate	An expression did not yield a definite Boolean.	NULL handling and the exact compared columns
Header totals do not reconcile	A line, discount or rounding rule disagrees with the invoice.	Line amounts, TotalNetHt and allocated residual
A diagram is not visible	Master is open, or the file is being viewed as XML.	Open the SSIS designer and the Data Flow inside ETL_SA_DW

Read the first specific error rather than only the final package-failure line. DTExec reports exit 0 for success and a nonzero result for failure. SQL validation uses THROW to produce explicit error messages. sql-validation.log may contain an aggregate NULL-elimination warning because unknown optional values are deliberately excluded from aggregates; that warning is not a failed assertion.

The implementation does not promise an all-warehouse rollback for every possible failure. Raw staging is refreshed and parts of an unsuccessful run may have completed. Fix the source/configuration problem and rerun Master to restore the intended final state. The finally blocks in verification restore its controlled fixtures, but do not provide protection from a forced operating-system termination.

The packages are intended for one coordinated local run at a time. Scheduling, multi-run concurrency control, production monitoring and backup policy are future operational work, not completed features.

34 Known limits and responsible interpretation

A completed integration means the implemented sources, transformations, schema and validation have run successfully. It does not mean the source is perfect, that every possible future change is covered, or that business owners have confirmed all inherited classifications.

Limit	Consequence
Stock capture date not established	DateKey remains NULL and no historical stock trend is claimed.
Missing product descriptions or thresholds	Explicit unknown values remain visible; no guessed spelling or default threshold is presented as fact.
Supplier credit notes excluded	Purchase amounts are net of discounts but not net of returns.
French holiday logic inherited	The business must confirm the relevant country calendar.
SCD type 1 dimension updates	Historical purchases use the current product family/name, not an old attribute version.
Full file staging refresh	This is a snapshot-based file integration, not real-time change data capture.
Local configuration and files	Another computer needs the runtime, source files and configured connections.
No published remote update	The working changes are local; GitHub publication was not requested.

No sales, receipt or supplier-payment fact is part of this final seven-table warehouse. Client and payment dimensions are retained because they are part of the group’s dimensional foundation and can support later facts. A fact is not required to connect to every available dimension.

The 67 passing checks establish the recorded behavior on the supplied dataset: faithful raw transfer, stable reruns, native loading, tested repairs, source reclassification handling and reconciliation. They do not replace a business review of source semantics or guarantee a grade.

35 Explain the project to the teacher

A coherent demonstration order

Start with the business questions: which products are below their stock threshold, and how much was purchased from each supplier by period? Show the architecture diagram, then the two real SQL databases. Define the grain of both facts before showing their measures.

Open Master to explain orchestration. Open one raw staging Data Flow. Then show the product or client dimension with Sort and Merge Join, followed by the supplier Lookup pattern. Finally show the stock and purchase fact flows, including their INSERT and UPDATE branches.

For purchases, explain the validated-invoice filter, comment exclusions, supplier unknown member, date conversion and global discount allocation. Show that each invoice's net line sum equals its header. Finish with row counts, a simple analysis query and the verification report.

Questions worth preparing

Question	Answer
Why have a Staging Area?	To preserve source evidence and separate receiving data from interpreting it.
Why are the counts different?	Source rows and dimension/fact rows have different grains and eligibility rules.
Why not use today as the stock date?	Processing time does not prove the date of the exported business snapshot.
Why are payment and client not in these facts?	They support other business processes; these two facts do not need those keys.
What happens on a second run?	Stable inputs produce identical final rows; new/changed keys follow INSERT/UPDATE.
Is everything visual SSIS?	Warehouse row loading is native Data Flows; SQL handles setup, documented source preparation and assertions.
What does a successful test not prove?	It does not recover missing business descriptions or certify all future production scenarios.

36 Implementation files and traceability

File or folder	Role
HealthRetailers_SSIS.sln and .dtproj	Open/build the SSIS project for SQL Server 2022
EL_SI_SA.dtsx	Raw staging import
ETL_SA_DW.dtsx	Four dimension and two fact Data Flows plus preparation
Master.dtsx	Staging, calendar, loading and validation entry point
sql/01_SA_Health.sql	Portable raw staging schema
sql/02_DW_Health.sql	Five dimension schemas
sql/03_alim_dim_temps.sql	Repeatable calendar generation used by Master
sql/04_dimension_sources.sql	Text helpers and product source procedure
sql/05_faits.sql	Two fact schemas, numeric helper, caches, views and preparation
sql/99_verifications.sql	Warehouse assertions embedded in Master
sql/07_analyses.sql	Read-only demonstration queries
scripts/configure.py and sources.json	Transport preparation and local configuration
scripts/build-facts.ps1 and FactGraphs.cs	Generate native fact components and paths through the SSIS API
scripts/layout-facts.py	Restore original dimension layouts and lay out the two new fact graphs
scripts/setup.ps1	Configure, install SQL objects and build
scripts/verify.ps1	Run the measured validation and regression suite

The guide’s diagrams are reconstructed from the actual package metadata and documented SQL behavior. Where a long flow is split into two explanatory diagrams, this is a presentation choice; the saved package still contains one connected Data Flow.

The reproducible source of this guide is kept in docs/complete-guide. The package and SQL files, not this prose, are the execution authority. The following appendices expose every final warehouse field, raw source field inventory, recorded check and principal technical term.

Appendix A Warehouse field dictionary 1

These are the implemented fields and types. PK means primary key, FK means foreign key. NULL is a permitted physical value; the final validation can impose a stronger business requirement.

Product dimension fields

SQL table dim_produit

Field	Type and key	Lineage and meaning
Product_Id	int PK NOT NULL	Source ProductStocks product identifier; one member per product.
Libelle	nvarchar(255) NULL	Chosen catalogue, line or event description; explicit unknown fallback required by QA.
Famille	nvarchar(100) NOT NULL	Catalogue family or NON RENSEIGNE.
Marque	nvarchar(100) NOT NULL	Catalogue brand, known prefix inference or NON RENSEIGNE.

Client dimension fields

SQL table dim_client

Field	Type and key	Lineage and meaning
Company_Id	int PK NOT NULL	Company identifier; includes -1 unknown and 2 anonymous.
Nom	nvarchar(200) NULL	Cleaned customer name, company fallback or explicit special-member label.
TypeClient	nvarchar(30) NOT NULL	Known customer classification or NON RENSEIGNE.
EstClientIdentifie	bit NOT NULL	0 for -1 and 2, otherwise 1 for retained customer identifiers.

Appendix A Warehouse field dictionary 2

These are the implemented fields and types. PK means primary key, FK means foreign key. NULL is a permitted physical value; the final validation can impose a stronger business requirement.

Supplier dimension fields

SQL table dim_fournisseur

Field	Type and key	Lineage and meaning
Fournisseur_Id	int PK NOT NULL	Company/supplier identifier; -1 is the unknown member.
Nom	nvarchar(200) NULL	Supplier-export or company name; explicit unknown fallback.

Payment mode dimension fields

SQL table dim_paiement

Field	Type and key	Lineage and meaning
PaymentMode_Id	int PK NOT NULL	Distinct source payment-mode identifier; -1 unknown.
PaymentModeName	nvarchar(50) NOT NULL	Readable mapping of the application payment-mode label.

Appendix A Warehouse field dictionary 3

These are the implemented fields and types. PK means primary key, FK means foreign key. NULL is a permitted physical value; the final validation can impose a stronger business requirement.

Calendar dimension fields

SQL table dim_temps

Field	Type and key	Lineage and meaning
DateKey	int PK NOT NULL	YYYYMMDD integer key.
Date	date NOT NULL	Calendar date from 2024-09-30 to 2026-09-29.
Annee	smallint NOT NULL	Calendar year.
Trimestre	tinyint NOT NULL	Quarter 1 to 4.
Mois	tinyint NOT NULL	Month 1 to 12.
SemaineISO	tinyint NOT NULL	ISO week number.
EstFerie	bit NOT NULL	Inherited French holiday indicator.

Current stock fact fields

SQL table fait_stock_actuel

Field	Type and key	Lineage and meaning
Product_Id	int PK FK NOT NULL	Current product snapshot key referencing dim_produit.
DateKey	int FK NULL	Optional reference to dim_temps; always NULL in this export-based version.
QuantiteActuelle	decimal(18,4) NOT NULL	Sum of source stock quantities at product grain.
QuantiteAlerte	decimal(18,4) NULL	Known consistent threshold, otherwise NULL.
EstSousSeuil	bit NULL	1 if quantity is below a known threshold; 0 if not; NULL if no threshold.

Appendix A Warehouse field dictionary 4

These are the implemented fields and types. PK means primary key, FK means foreign key. NULL is a permitted physical value; the final validation can impose a stronger business requirement.

Purchase fact fields

SQL table fait_achats

Field	Type and key	Lineage and meaning
Id	bigint PK NOT NULL	Unique source DocumentItem line identifier.
Document_Id	bigint NOT NULL	Invoice identifier retained as a document reference.
DateKey	int FK NOT NULL	Invoice date key referencing dim_temps.
Product_Id	int FK NOT NULL	Product reference from the line’s stock identifier.
Fournisseur_Id	int FK NOT NULL	Supplier reference from the invoice transaction; -1 when unassigned.
QuantiteAchetee	decimal(18,4) NOT NULL	Source line quantity.
MontantAchat_HT	decimal(18,4) NOT NULL	Line HT after its allocated document discount and residual correction.

Appendix B Raw source fields 1

Every listed field is retained as raw Unicode text. Names match the staging schema; source/target ordering and connections are configured in scripts/sources.json. UsersSettings.Value uses NVARCHAR(MAX); the other raw fields use NVARCHAR(4000).

Clients SA

File Clients - 24112025081756 (3).csv | 93 rows | separator ';' | decoding utf-8-sig

Fields: Code, Nom, Adresse, Code postal, Ville, Pays, E-Mail, Site Web, Solde, Solde initial, Registre de commerce.

CompaniesList SA

File CompaniesList.csv | 156 rows | separator ';' | decoding cp1252

Fields: Id, IsRemoved, CompanyType_Id, Name, IsGuest, Code, Id3, Id4, Id5, Id6, Id7, telephone, email, Extra1, Extra2, Extra3, Extra4, Extra5.

DocumentItem SA

File documentItemm.csv | 38705 rows | separator ';' | decoding utf-8-sig

Fields: Id, DisplayOrder, Description, Quantity, PrixHt, Tva, PrixTotalHt, PrixTotalTtc, DiscountPourcentage, DiscountAmount, Document_Id, ProductStock_Id, ProductPackage_Id, TotalTva, IncludedTaxes, TotalTaxes, SumPrixNetHt, SumQuantityUnit, SumDiscountTotal, SumPrixTotalHt, SumTotalTva, SumIncludedTaxes, SumTotalTaxes, SumPrixTotalTtc, PrixTtc, ForcedPrixTtc, SumPrixNetTtc, Extra, PrixTvc, ProductStockLocation_Id, BenTaxCategory_Id, NerTaxCategory_Id, TtcDiscountPercentage, TtcDiscountAmount.

Documents SA

File Documents.csv | 12542 rows | separator ',' | decoding cp1252

Fields: Id, DocumentNumber, DocumentDate, State, TotalHt, TotalNetHt, DiscountPercentage, TotalTva, TotalTtc, Timbre, FinalPrice, IsDelivered, Observation, DocumentType_Id, Transaction_Id, PaymentMode_Id, ProductPriceName_Id, DefaultStock_Id, UserComment, Attachments_Id, DiscountAmount, TotalDiscount, IncludedTaxes, TotalTaxes, TotalExtraTaxes, CreatedDate, CreatedBy, ModifiedDate, ModifiedBy, ValidatedDate, ValidatedBy, Extra, TotalRetenueSurHt, TotalRetenueSurTva, TotalRetenueSurTtc.

Appendix B Raw source fields 2

Every listed field is retained as raw Unicode text. Names match the staging schema; source/target ordering and connections are configured in scripts/sources.json. UsersSettings.Value uses NVARCHAR(MAX); the other raw fields use NVARCHAR(4000).

DocumentTypes SA

File DocumentTypes.csv | 15 rows | separator ';' | decoding utf-8-sig

Fields: Id, Name, Label, IconPath, ReportBottomMsg, CanValidate, CanCreate, StockFactor, AllowBlackStock, IsSellType, IsPayable, PayementPriority, StockPriority, IsRefund, IsDeliverable, IsEnabled.

Events SA

File Events.csv | 82585 rows | separator ';' | decoding utf-8-sig

Fields: Id, EntityId, EntityType, DependentEntityId, DependentEntityType, DependentEntity2Id, DependentEntity2Type, EventType, Data, Reason, Timestamp, UserId.

Fournisseurs SA

File Fournisseurs - 24112025081816 (2).csv | 118 rows | separator ';' | decoding utf-8-sig

Fields: Code, Nom, Adresse, Code postal, Ville, Pays, E-Mail, Site Web, Solde, Solde initial.

PaymentsList SA

File PaymentsList.csv | 10280 rows | separator ';' | decoding utf-8-sig

Fields: Id, IsRemoved, Company_Id, Amount, PaymentDate, Reference, Description, PaymentMode_Id, PaymentMethodName, Extra.

Appendix B Raw source fields 3

Every listed field is retained as raw Unicode text. Names match the staging schema; source/target ordering and connections are configured in scripts/sources.json. UsersSettings.Value uses NVARCHAR(MAX); the other raw fields use NVARCHAR(4000).

Payment SA

File Payment.csv | 10899 rows | separator ';' | decoding cp1252

Fields: Id, Amount, UsedAmount, PaymentDate, Description, Company_Id, PaymentMode_Id, Reference, Extra.

ProductBrands SA

File ProductBrands.csv | 212 rows | separator ',' | decoding utf-8-sig

Fields: Id, BrandName.

ProductStocks SA

File ProductStocks.csv | 6842 rows | separator ';' | decoding utf-8-sig

Fields: Id, Quantity, Stock_Id, Product_Id, TotalQuantity.

Produits SA

File Produits - 24112025081625 (2).csv | 7800 rows | separator ';' | decoding utf-8-sig

Fields: Code, Libellé, Libellé ticket de caisse, Famille, Marque, Prix d'achat HT, Prix de vente HT, Tva, Prix de vente TTC, Quantité, Poids (Kg), Unité, Désignation, Observations, Quantité d'Alerte.

Appendix B Raw source fields 4

Every listed field is retained as raw Unicode text. Names match the staging schema; source/target ordering and connections are configured in scripts/sources.json. UsersSettings.Value uses NVARCHAR(MAX); the other raw fields use NVARCHAR(4000).

RelatedDocuments SA

File RelatedDocuments.csv | 864 rows | separator ';' | decoding utf-8-sig

Fields: Id, Relation, Document1_Id, Document2_Id.

StockMovements SA

File StockMovements.csv | 49609 rows | separator ';' | decoding utf-8-sig

Fields: Id, DocumentId, MovementDate, Quantity, CurrentBalance, Description, ProductStock_Id, User_Id, Extra, ProductStockLocation_Id, CurrentLocationBalance.

TransactionPayments SA

File TransactionPayments.csv | 10962 rows | separator ',' | decoding utf-8-sig

Fields: Id, Amount, Transaction_Id, Payment_Id.

Transactions SA

File Transactions.csv | 12523 rows | separator ';' | decoding utf-8-sig

Fields: Id, IsPayed, PayedAmount, IsDestocked, IsRestocked, Company_Id, PaymentDocument_Id, RefundDocument_Id, DestockDocument_Id, RestockDocument_Id, BillingAddressId, ShippingAddressId.

Appendix B Raw source fields 5

Every listed field is retained as raw Unicode text. Names match the staging schema; source/target ordering and connections are configured in scripts/sources.json. UsersSettings.Value uses NVARCHAR(MAX); the other raw fields use NVARCHAR(4000).

UsersSettings SA

File users_settings_csv.csv | 95 rows | separator ';' | decoding utf-8-sig

Fields: Id, SettingName, Value.

Technical staging tables

The 17 raw tables are supplemented by three technical tables. These prepared or diagnostic records do not replace the original source evidence.

Object	Fields and types
Rejets_Faits	Fait nvarchar(40) NOT NULL; SourceId bigint NULL; Motif nvarchar(100) NOT NULL. Stores the excluded candidate line and its reason.
CatalogueSeuils_Faits	Code nvarchar(200) NULL; Libelle nvarchar(255) NULL under CI_AI collation; Seuil decimal(18,4) NULL. Indexed by code and label.
CodesProduits_Faits	Product_Id int PRIMARY KEY NOT NULL; Code nvarchar(200) NULL. One latest product-event code per product identifier.

Candidate view outputs

v_source_stock returns Product_Id, QuantiteActuelle and QuantiteAlerte. The Data Flow adds DateKey and EstSousSeuil.

v_source_achats returns Id, Document_Id, DocumentDate, IsValidated, Product_Id, Fournisseur_Id, Quantity, TotalHt, RemiseDocument_HT and MontantNet_HT. Intermediate invoice totals and row numbers are used within the SQL calculation without changing the invoice-line grain.

Both caches are rebuilt by usp_preparer_faits. Rejets_Faits is cleared for the next run after mandatory-value guards succeed, then populated by the purchase exclusion branch.

Appendix C Recorded checks 1

Exact check labels from reports/verification-results.json. The final entry records the seven loaded table counts; the aggregate SQL check includes row/measure reconciliation and trusted foreign keys.

No	Recorded check	Result
1	SSIS master.log	PASS
2	Stage count Clients_SA	PASS
3	Lossless Unicode Clients_SA	PASS
4	Stage count CompaniesList_SA	PASS
5	Lossless Unicode CompaniesList_SA	PASS
6	Stage count DocumentItem_SA	PASS
7	Lossless Unicode DocumentItem_SA	PASS
8	Stage count Documents_SA	PASS
9	Lossless Unicode Documents_SA	PASS
10	Stage count DocumentTypes_SA	PASS
11	Lossless Unicode DocumentTypes_SA	PASS
12	Stage count Events_SA	PASS
13	Lossless Unicode Events_SA	PASS
14	Stage count Fournisseurs_SA	PASS
15	Lossless Unicode Fournisseurs_SA	PASS
16	Stage count PaymentsList_SA	PASS
17	Lossless Unicode PaymentsList_SA	PASS
18	Stage count Payment_SA	PASS

Appendix C Recorded checks 2

Exact check labels from reports/verification-results.json. The final entry records the seven loaded table counts; the aggregate SQL check includes row/measure reconciliation and trusted foreign keys.

No	Recorded check	Result
19	Lossless Unicode Payment_SA	PASS
20	Stage count ProductBrands_SA	PASS
21	Lossless Unicode ProductBrands_SA	PASS
22	Stage count ProductStocks_SA	PASS
23	Lossless Unicode ProductStocks_SA	PASS
24	Stage count Produits_SA	PASS
25	Lossless Unicode Produits_SA	PASS
26	Stage count RelatedDocuments_SA	PASS
27	Lossless Unicode RelatedDocuments_SA	PASS
28	Stage count StockMovements_SA	PASS
29	Lossless Unicode StockMovements_SA	PASS
30	Stage count TransactionPayments_SA	PASS
31	Lossless Unicode TransactionPayments_SA	PASS
32	Stage count Transactions_SA	PASS
33	Lossless Unicode Transactions_SA	PASS
34	Stage count UsersSettings_SA	PASS
35	Lossless Unicode UsersSettings_SA	PASS
36	SSIS master-rerun.log	PASS

Appendix C Recorded checks 3

Exact check labels from reports/verification-results.json. The final entry records the seven loaded table counts; the aggregate SQL check includes row/measure reconciliation and trusted foreign keys.

No	Recorded check	Result
37	Idempotency dim_client	PASS
38	Idempotency dim_fournisseur	PASS
39	Idempotency dim_produit	PASS
40	Idempotency dim_paiement	PASS
41	Idempotency dim_temps	PASS
42	Idempotency fait_stock_actuel	PASS
43	Idempotency fait_achats	PASS
44	SSIS dimension-null-and-update.log	PASS
45	NULL/change repair dim_client	PASS
46	NULL/change repair dim_fournisseur	PASS
47	NULL/change repair dim_produit	PASS
48	NULL/change repair dim_paiement	PASS
49	NULL/change repair dim_temps	PASS
50	NULL/change repair fait_stock_actuel	PASS
51	NULL/change repair fait_achats	PASS
52	Stock NULL fixture has known threshold	PASS
53	SSIS facts-update.log	PASS
54	Fact UPDATE repair fait_stock_actuel	PASS

Appendix C Recorded checks 4

Exact check labels from reports/verification-results.json. The final entry records the seven loaded table counts; the aggregate SQL check includes row/measure reconciliation and trusted foreign keys.

No	Recorded check	Result
55	Fact UPDATE repair fait_achats	PASS
56	SSIS facts-source-reclassification.log	PASS
57	Remove reclassified invoice facts	PASS
58	SSIS facts-source-restored.log	PASS
59	Restored source and warehouse dim_client	PASS
60	Restored source and warehouse dim_fournisseur	PASS
61	Restored source and warehouse dim_produit	PASS
62	Restored source and warehouse dim_paiement	PASS
63	Restored source and warehouse dim_temps	PASS
64	Restored source and warehouse fait_stock_actuel	PASS
65	Restored source and warehouse fait_achats	PASS
66	SQL reconciliation and trusted foreign keys	PASS
67	Five dimensions and two facts	PASS

Appendix D Technical terms 1

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
Aggregate	Combine several values at a chosen grain, for example SUM of stock quantities per product.
API	Application programming interface. The authoring scripts use the SSIS API to create native package objects programmatically.
Assertion	An executable condition that must be true. The final SQL assertions throw an error when a required condition fails.
Bigint	SQL Server integer type used here for document and invoice-line identifiers.
Binary collation	Text comparison based on exact character values. Used for reliable corruption-character detection.
Bit	SQL Boolean-like storage with values 0 or 1; a nullable bit can also be NULL.
BOM	Byte order mark, an optional encoding marker at the start of a text file. The decoding logic handles UTF8 with an optional BOM.
Boolean	A true-or-false value. SSIS Conditional Split expressions must produce a definite Boolean rather than NULL.
Business key	Identifier originating in the business application, such as Product_Id. Existing business identifiers are used as dimension keys here.
Cache	Prepared data kept for repeated lookup. The two staging caches are rebuilt each load and do not replace the raw evidence.
Cardinality	How many records can participate in a relationship. Stock currently permits at most one fact row per product.
CI AI	Case-insensitive and accent-insensitive collation behavior. Used for matching labels without requiring identical case or accent marks.

Appendix D Technical terms 2

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
Clone	A local copy of a Git repository, including its history and available remote references.
Collation	Rules SQL Server applies when comparing or sorting text, including case and accent sensitivity.
Column	A named field shared by the rows of a table, such as Quantity.
Conditional Split	SSIS transformation that routes a row according to a Boolean expression. It does not itself change the database.
Conformed dimension	A shared, consistently defined dimension. Product and date are reused by the final facts.
Connection manager	SSIS definition of how to access a file or database. Project managers are stored as .conmgr files.
Control Flow	Package-level task orchestration. Its arrows establish execution conditions rather than carrying table rows.
CP1252	Windows character encoding used by three selected source files in the recorded manifest.
Credit note	Document that reverses or adjusts an invoice. Supplier credit notes are type 23 and are excluded from the delivered purchase fact.
CSV	Delimited text records. These exports use commas or semicolons with configured quotation rules.
CTE	Common table expression, a named query expression used within a SQL statement. The purchase view uses CTEs to explain staged calculations.
Data Flow	The SSIS row-processing graph containing sources, transformations and destinations.

Appendix D Technical terms 3

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
Data lineage	The trace from an output field back to its source fields and transformation rules.
Data warehouse	Database organized to support business analysis. The final database has five dimension tables and two fact tables.
Database	Organized collection of tables and other objects managed by the SQL Server engine.
Database instance	Running SQL Server service addressed by a server name. The configured instance is localhost.
DateKey	Integer calendar key in YYYYMMDD format. It is different from a free-text date label.
DDL	Data definition language, the SQL statements that create or change schemas, tables, keys, views and routines.
Decimal precision and scale	decimal(18,4) stores 18 total digits, four after the decimal point. It avoids binary floating-point storage for the fact measures.
Delimiter	Character separating CSV fields, a comma or semicolon in the selected exports.
Derived Column	SSIS transformation that adds or replaces a value using an expression, such as date key, selection reason or net amount.
Dimension	Descriptive table used to analyse facts, for example product family or supplier name.
Draft	A source document that is not validated. State 0 invoices do not contribute purchase facts.
DT I4 and DT BOOL	SSIS expression types: a four-byte integer and a Boolean. Used by NULL(DT_I4) and NULL(DT_BOOL).

Appendix D Technical terms 4

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
DT NUMERIC and DT WSTR	SSIS exact numeric and Unicode string types used in Derived Column casts.
DTExec	Microsoft command-line utility used to execute the built SSIS project and return an execution result.
Encoding	Rule mapping characters to file bytes. UTF8, UTF16 and CP1252 are different encodings encountered in this project.
ETL	Extract, Transform, Load. Reading the exports, interpreting their fields and storing the warehouse records form this pipeline.
EXCEPT	SQL set-difference operator. Used in both directions to find source-only and warehouse-only fact rows.
Execute Package task	Control Flow task that calls another .dtsx package from the project.
Execute SQL task	Control Flow task that runs SQL, such as calendar generation, preparation or validation.
Fact constellation	Several fact tables sharing conformed dimensions. This describes the product/date sharing in the small final model.
Fact table	Table storing observations and measures at an explicit grain, with references to relevant dimensions.
File extensions	Suffixes identifying artifact formats: .sln solution, .dtproj SSIS project, .dtsx package, .conmgr connection and .ispac compiled project.
Fixture	A controlled temporary change introduced by a test, then restored. It is not a new production business record.
Flat File Source	SSIS source component that reads delimited text using a connection manager.

Appendix D Technical terms 5

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
Foreign key	Reference that requires a matching parent key, unless the referencing column permits NULL.
GO	Batch separator recognized by tools such as sqlcmd and SSMS. It is not a SQL instruction to send unchanged inside an Execute SQL task.
Grain	The precise level of detail of one row, such as one invoice line or one current product stock balance.
Hash	A compact deterministic fingerprint of content. It is used for comparison, not for recovering missing source characters.
HT	Hors taxes, the amount excluding tax. MontantAchat_HT is net of document discounts in this implementation.
HTML	Markup embedded in some descriptions. Tags and selected entities are cleaned where product source logic requires it.
Idempotence	Repeated execution with the same inputs leads to the same final business-table contents.
Index	SQL access structure that helps repeated searches. The technical code cache has a primary-key index and the threshold cache has lookup indexes.
Integration Services runtime	The installed SSIS execution components. A database engine connection alone is not enough to execute every SSIS package.
Invoice header	Document-level record containing date, supplier transaction link, status and totals.
Invoice line	A product or comment row associated with a document. Its source Id defines the purchase fact key.
JSON	Structured text with named values. Product events contain id, cde and lbl evidence.

Appendix D Technical terms 6

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
Lookup	SSIS key search against a reference dataset. No-match can be redirected for insertion or configured to fail for a required dimension.
Manifest	Inventory of source files, columns, expected counts, encodings and fingerprints.
Measure	Quantitative fact value that can be analysed at its defined grain. Additivity depends on the measure and dimensions.
Merge Join	SSIS transformation that combines two sorted rowsets by a join key. Used by the retained product and client dimension flows.
Metadata	Information describing data, such as column names, data types and pipeline component connections.
Mojibake	Text damaged by a wrong encoding interpretation, sometimes repairable from a known byte/character sequence.
Multiset	Collection that retains repeated occurrences. The cell-hash comparison keeps duplicate row counts while ignoring ordering.
Natural key	Another name for a business-origin identifier, contrasted with an independently generated surrogate key.
NULL	Missing or unknown SQL value. It is different from zero; normal comparisons involving NULL may yield unknown.
Nullable	A column is nullable when its schema permits NULL. Business validation may impose an additional nonempty-value requirement.
NULLIF	SQL function returning NULL when two expressions are equal. It converts trimmed empty identifier strings to missing values before numeric conversion.
NVARCHAR	SQL Server Unicode text type. The raw staging uses it to preserve decoded characters.

Appendix D Technical terms 7

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
OLE DB	Database access interface used by native SSIS sources, destinations, lookups and commands.
OLE DB Command	SSIS transformation that executes a parameterized SQL command for each input row. Used for the explicit UPDATE paths.
OLE DB Destination	SSIS component writing input rows into a SQL target table. Fast load is used with constraint checking.
OLE DB Source	SSIS component that reads a table, view or SQL query and produces pipeline rows.
Orphan reference	Fact key with no corresponding dimension row. Required Lookups and foreign keys prevent these from being silently loaded.
Parameter	Value supplied separately to a command. OLE DB Command uses placeholders and mapped row values for updates.
Parsing	Reading a representation according to its structure, such as CSV quoting rules or JSON field names.
PK	Primary key. The unique non-NULL identifier of a table row.
Precedence constraint	Control Flow connector determining whether a task may run, normally after predecessor success here.
Proportional allocation	Distribute a document-level amount according to line shares. Used for global purchase discounts.
Query	Request to retrieve or calculate data, usually expressed in SQL here.
Reconciliation	Compare source and loaded keys or measures so differences are explicit. Invoice totals are reconciled separately.

Appendix D Technical terms 8

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
Referential integrity	Consistency between referencing fact keys and their parent dimension keys.
Replacement character	U+FFFD marker indicating a character could not be represented during an earlier decoding step. It is not the original missing letter.
Rounding residual	Difference between a required total and the sum of separately rounded parts. Assigned to one deterministic commercial line here.
Row	One record in a table or SSIS pipeline. Its business meaning is determined by the grain.
ROW NUMBER	SQL window function that numbers rows within a group. It chooses latest events and the deterministic residual line.
SCD type 1	Slowly Changing Dimension behavior that overwrites an old attribute rather than keeping an attribute-history version.
Schema	The defined structure of database objects and fields. The dbo schema namespace contains the final tables.
SDK	Software development kit. Installed SSIS assemblies provide the authoring objects used by the setup helpers.
SHA256	Hash algorithm used for original file bytes, row values and table comparisons in the recorded checks.
Snapshot	An observation of a state at a point in time. The supplied stock capture date is unknown; the fact stores current source state only.
Sort	SSIS transformation called Trier that orders input rows before the client/product Merge Joins.
SQL	Structured Query Language, used to define objects, retrieve candidates, prepare sources, update records and validate results.

Appendix D Technical terms 9

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
SQLCMD Mode	SSMS mode that accepts script variables and batch tooling conventions used by the setup scripts.
SSIS	SQL Server Integration Services, the package and Data Flow integration platform used by this project.
SSMS	SQL Server Management Studio, a database administration and querying client.
Staging Area	Intermediate data layer between source exports and warehouse tables. It holds raw rows and technical preparation here.
Star schema	A fact table linked to descriptive dimensions. Each final fact forms a small star.
Stored procedure	Named SQL routine installed inside a database. Product matching and fact preparation are procedures here.
Surrogate key	Artificial warehouse-generated identifier independent of the source business key. This version retains business keys instead.
Table	A stored set of rows sharing named typed columns. Views expose queries; tables store the records.
Ternary expression	An expression choosing one value when a condition is true and another when false, written condition ? yes : no in SSIS.
Text qualifier	Character identifying a quoted CSV cell. Double quotation marks protect embedded delimiters and JSON quotes.
Transaction	A coordinated database work unit in SQL, or a business transaction in the source application. Transactions_SA stores the latter; the complete ETL is not promised to roll back as one atomic SQL transaction.
Trusted foreign key	Constraint whose existing data has also been validated. Fast-load constraint checking preserves this state.

Appendix D Technical terms 10

Definitions describe the term and, where useful, its role in this implementation.

Term	Definition
TRY CAST and TRY CONVERT	SQL conversions returning NULL on a conversion failure, allowing an explicit subsequent quality decision.
TTC	Toutes taxes comprises, a tax-inclusive amount. It is not the purchase fact's HT measure.
Unicode	Character system supporting accented and multilingual text; UTF8 and UTF16 are encodings used to represent it.
Unknown member	A real dimension row representing an unassigned reference, typically key -1 here. It differs from inventing an actual supplier.
Upsert	Insert a missing key or update the values of an existing key.
Validation	Executable checking of defined rules. It is not proof that every source business value is known or that all future scenarios are covered.
View	Saved SQL query definition that exposes candidate rows without itself storing a separate copy of those rows.
Window function	SQL calculation over related rows while retaining the individual row grain, such as an invoice total beside each line.
Windows Authentication	SQL connection using the Windows identity rather than embedding a SQL username and password.
Working branch	Local Git line of work. The final changes remain on codex/fix-five-dimensions unless deliberately published later.

Appendix E Evidence and reading order

Primary implementation base: https://github.com/amal-mh/HealthRetailers_DW at upstream commit 3d52d4f48c66da6eccee05ee1050188ae1a4af6d. Original wider project: https://github.com/Youssef-ben-ali/DataWarehouseProject_3AI4. The final runtime project is the local corrected Amal-based copy, not an assertion about either remote repository's current deployed state.

Current local project: C:\Users\firas\OneDrive\Documents\ssis\HealthRetailers_DW_amal. Final source code and packages are referenced by their relative filenames throughout this guide. The adjacent original project remains a separate historical implementation.

For a first reading, follow chapters 1 to 11 to understand purpose and structure, then the relevant dimension and fact chapters. Use chapters 28 and 29 for evidence, chapters 30 to 33 for operation, and the appendices for exact fields and vocabulary.

This guide is an English explanation of the final implemented scope, assembled from package XML, SQL definitions, the source manifest and recorded validation results. Diagrams are reconstructed documentation rather than screenshots. Fictional worked examples are explicitly labelled; reported business totals are measured.

No production database credentials, private customer-row listings or invented missing source descriptions are needed to understand the process. The guide does not alter the packages, source data or SQL databases. The operational scripts remain the authority for execution.